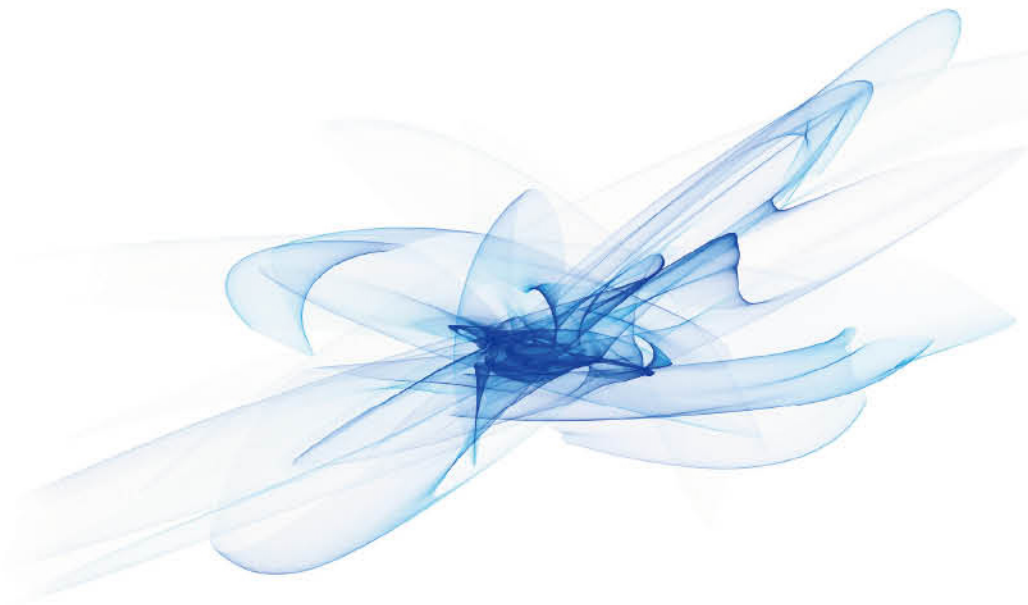


从多个角度全面讲解Storm实时数据处理技术和最佳实践，为快速掌握并灵活应用Storm提供实用指南

从实际问题出发，系统介绍Storm的基本应用、多语言特性、完整业务系统实现和产品交付的最佳实践方法；从产品持续交付角度，分析并实践集成、测试和交付的所有步骤



Storm Real-Time Processing Cookbook

# Storm实时数据处理

(澳) Quinton Anderson 著

卢誉声◎译



机械工业出版社  
China Machine Press

大数据技术丛书

# Storm 实时数据处理

Storm Real-Time Processing Cookbook

(澳) Quinton Anderson 著

卢誉声 译



机械工业出版社  
China Machine Press

## 图书在版编目 (CIP) 数据

Storm 实时数据处理 / (澳) 安德森 (Anderson, Q.) 著; 卢誉声译. —北京: 机械工业出版社, 2014.6

(大数据技术丛书)

书名原文: Storm Real-Time Processing Cookbook

ISBN 978-7-111-46663-5

I. S… II. ① 安… ② 卢… III. 数据处理软件 IV. TP274

中国版本图书馆 CIP 数据核字 (2014) 第 103057 号

本书版权登记号: 图字: 01-2013-7570

Quinton Anderson: Storm Real-Time Processing Cookbook (ISBN: 978-1-78216-442-5).

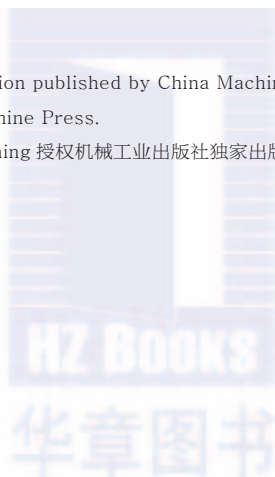
Copyright © 2013 Packt Publishing. First published in the English language under the title “Storm Real-Time Processing Cookbook”.

All rights reserved.

Chinese simplified language edition published by China Machine Press.

Copyright © 2014 by China Machine Press.

本书中文简体字版由 Packt Publishing 授权机械工业出版社独家出版。未经出版者书面许可, 不得以任何方式复制或抄袭本书内容。



## Storm 实时数据处理

[澳] Quinton Anderson 著

出版发行: 机械工业出版社 (北京市西城区百万庄大街 22 号 邮政编码: 100037)

责任编辑: 高婧雅

责任校对: 殷虹

印刷:

版次: 2014 年 6 月第 1 版第 1 次印刷

开本: 186mm × 240mm 1/16

印张: 12.75

书号: ISBN 978-7-111-46663-5

定价: 49.00 元

凡购本书, 如有缺页、倒页、脱页, 由本社发行部调换

客服热线: (010) 88378991 88361066

投稿热线: (010) 88379604

购书热线: (010) 68326294 88379649 68995259

读者信箱: hzsj@hzbook.com

版权所有·侵权必究

封底无防伪标均为盗版

本书法律顾问: 北京大成律师事务所 韩光 / 邹晓东

## 译者序

随着互联网业务数据规模的急剧增加，人们处理和使用数据的模式已然发生了天翻地覆的变化，传统的技术架构越来越无法适应当今海量数据处理的需求。MapReduce、Hadoop 以及一些相关技术的出现使得我们能处理的数据量比以前要多得多，这类技术解决了我们面对海量数据时的措手不及，也在一定程度上缓解了传统技术架构过时的问题。

但是，随着业务数据规模的爆炸式增长和对数据实时处理能力的需求越来越高，原本承载着海量数据处理任务的 Hadoop 在实时计算处理方面越发显得乏力。原因很简单，像 Hadoop 使用的 MapReduce 这样的数据处理技术，其设计初衷并不是为了满足实时计算的需求。那么就目前来说，有没有什么行之有效的办法能简单地将 Hadoop 转换成实时计算系统呢？

这个问题的答案可能令人略感失望：没有。Hadoop 作为批处理系统，与实时处理系统在需求上存在着本质的区别。要做到实时性，不仅需要及时地推送数据以便处理，还要将数据划分成尽可能小的单位，而 HDFS 存储推送数据的能力已经远不能满足实时性的需求。另外，Hadoop 还存在配置、一致性和可伸缩性方面的问题。

那么问题来了，怎么才能构建出一个可靠的实时处理系统呢？

答案是 Storm。

从整体架构上看，Storm 和 Hadoop 非常类似。Storm 从架构基础本身就实现了实时计算和数据处理保序的功能，而且从概念上看，Storm 秉承了许多 Hadoop 的概念、术语和操作方法，因此如果你对 Hadoop 非常熟悉，那么将 Storm 与 Hadoop 集成也不是什么难事。通过 Storm Trident 提供的高级抽象元语，你可以像 Hadoop Cascading 简化并行批处理那样简化并行实时数据处理。

我本人在实时计算服务器开发方面具有一定的经验，对大数据处理解决方案十分感兴趣，也对相关技术有一些了解，并且有幸承担了本书的翻译工作。本书从多个角度解析了有关 Storm 的最佳实践，无论是从最基本的应用、多语言特性、完整业务系统的实现，还是最终交付至产品环境，本书都给出了翔实的最佳实践方法。不仅如此，本书还从产品持续交付的角度，分析并实践了集成、测试和交付的所有步骤，让人受益匪浅。相信你会和我一样，通过阅读本书，能对 Storm 本身及构建成熟实时计算系统的方法有更进一步的了解。

翻译本书对我来说其实也是一个渐进学习的过程，书中提及了大量方法和工具的应用，

让我从零散的概念分支逐渐有了清晰的知识主干，形成了系统的知识点。通过对本书的翻译，我对这个领域的内容有了更加深刻的理解。这也是我翻译此书最大的收获之一。在翻译过程中，我不仅查阅了大量国内外的相关资料，还与原书作者进行了深入的沟通，力求做到专业词汇准确权威，书中内容正确。

在翻译过程中我得到了很多人的帮助，特在此一一感谢。首先是我的家人，你们是我学习和前进的动力。感谢鲁昌华教授，在我的成长道路上给予了很大的支持和鼓励。感谢我在思科系统（中国）研发的同事们，在我的学习工作中给予了很大帮助。感谢我的好友金柳颀，感谢你在翻译过程中的通力合作以及在技术问题上的共同探讨。还要感谢机械工业出版社华章公司对我的信任与支持。

现在我怀着期盼和忐忑的心情将这本译著呈献给大家，我渴望得到您的认可，更渴望和您成为朋友。如果您有任何问题和建议，请与我联系（[samblg@me.com](mailto:samblg@me.com)）。让我们一起探讨，共同进步。

卢誉声



# 前言

开源已经在许多方面从根本上改变了软件的原有面貌。在很多应用环境中，人们都会争论使用开源带来的好处和坏处，主要体现在支持、风险以及总体拥有成本等方面。开源在某些领域比其他领域流行，比如在研究机构中就比在大型金融服务提供商中应用得多。在某些新兴领域，比如 Web 服务供应商、内容供应商以及社交网络等，开源软件占据主导地位。其原因是多方面的，其中成本是一个非常大的因素。怎么说呢？如果方案要上升到网络规模，那么一般会应用“大数据”解决方案，以期获得更好的效果。凭借极佳的可用性，这些解决方案每秒处理数百万条请求，同时通过各式各样的服务为客户提供定制体验。

这种规模的系统设计需要我们用不同的方式思考问题，采用不同的架构解决方案，而且要了解什么时候应该接受系统的复杂性以及什么时候应该避免这种复杂性。作为行业中人，我们应该掌握这种大规模批处理系统的设计方法。由 MapReduce、Bulk Synchronous Parallel 以及其他计算范式衍生而来的大规模计算集群已经广泛实施，并且得到了很好的理解。开源掀起了创新浪潮，并推动其发展，而即使是顶级软件开发商也只能努力将 Hadoop 集成到自家的技术架构中，更别提试图与开源竞争了。

然而，客户是永不满足的，他们想要更多的数据、更多的服务、更多的价值、更多的便利，而且现在就要，并希望成本更低。随着数据量的增加，对实时响应时间的需求也在提高。专注于实时性、规模化的计算平台新时代已经来临。这带来了许多新的挑战，不管是理论上还是实践上都具有挑战性。

本书将帮助你掌握一个平台：Storm 处理系统。Storm 处理系统是一个开源的、实时的计算平台，最初由社交分析公司 Backtype 的 Nathan Marz 编写，后来被 Twitter 收购，并作为开源软件发布。从那时起，该平台茁壮成长，目前已经成长为一个日益扩大的开源社区：拥有众多用户、贡献者以及产品页面上的成功案例。写这篇前言的时候，该项目在 GitHub 上拥有超过 6000 个星、3000 多个 Twitter 粉丝，每个节点每秒可以完成超过 100 万次交易，而且有近 80 名相关用户在使用 Storm 产品实例。这些数字极其可观。此外，在本书的结尾你会发现，无论你用哪种语言（与你的思维方式以及解决方案交付方式相一致的语言）研发系统，以 Storm 平台为基础都会非常愉快。

本书通过一系列实例指导你学习 Storm。书中的例子源于实战用例，并随着内容的展开

介绍各种概念。此外，本书旨在围绕 Storm 技术促进 DevOps<sup>⊖</sup>实践，使读者能够开发 Storm 解决方案，同时可靠地交付具有价值的产品。

## Storm 处理系统简介

开源项目普遍存在缺乏文档的问题，但 Storm 并不存在这个问题，其项目有着高质量且编写良好的文档，由活跃的用户社区提供支持。本书并不想照本宣科，而是通过丰富的例子对现有资料进行补充，在必要的时候辅以概念和理论的探讨。强烈建议读者在阅读第 1 章之前，花些时间阅读 Storm 的入门文档，具体有以下几个网页：

- ❑ <https://github.com/nathanmarz/storm/wiki/Rationale>;
- ❑ <https://github.com/nathanmarz/storm/wiki/Concepts>;
- ❑ <https://github.com/nathanmarz/storm/wiki/Understanding-the-parallelism-of-a-Storm-topology>。

## 章节介绍

- 第 1 章 介绍搭建 Storm 本地开发环境的方法，包括所需工具和推荐的开发流程。
- 第 2 章 教你创建一个日志流处理解决方案、基本的统计面板和日志搜索功能。
- 第 3 章 介绍 Trident，Trident 是基于 Storm 元语进行实时计算的一类更高级的处理数据流的抽象元语，支持高效的企业数据管道。
- 第 4 章 介绍实现分布式远程过程调用的方法。
- 第 5 章 介绍在不同的编程语言中实现 Topology 的方法，并在现有技术基础上增加一些新的技术。
- 第 6 章 介绍 Storm 与 Hadoop 的集成方法，创建一个完整的 Lambda 架构。
- 第 7 章 简要介绍机器学习，讨论多种基于 Storm 实时处理项目的实现方法。
- 第 8 章 介绍搭建持续交付流水线和可靠部署 Storm 集群到产品环境的方法。
- 第 9 章 介绍多种在 Amazon 云计算平台中自动化配置 Storm 集群的方法。

## 阅读前提

本书采用 Ubuntu 或 Debian 操作系统作为基本的开发环境。第 1 章会介绍如何配置其余

---

⊖ DevOps是一组过程、方法和系统的统称，用于促进开发、技术运营和质量保障部门之间的沟通、协作与整合。DevOps的目的是实现业务敏捷，它将敏捷开发扩展到了运维上，通过快速、反应灵敏且稳定的业务运维，使其能够与开发过程的创新保持同步，实现真正的IT融合。——译者注

必备的工具。如果你不能使用 Ubuntu 作为开发用的操作系统，任何基于 \*Nix 的操作系统都是可以的，因为书中所有示例都以 bash 命令行界面为基础构建。

## 读者对象

如果你想学习实时处理技术，或想了解通过 Storm 实现实时处理的方法，那么本书非常适合你。本书假定你是一名 Java 开发者，要是你还具备 Clojure、C++ 和 Ruby 的开发经验，那将是锦上添花的事情，不过那并不是必需的。如果你还了解一些 Hadoop 或类似的技术就更好了。

## 本书版式约定

在本书中，读者会发现针对不同信息类型的文本样式。下面是这些样式的示例和解释。

代码段版式如下所示：

```
<repositories>

  <repository>
    <id>github-releases</id>
    <url>http://oss.sonatype.org/content/repositories
      /github-releases/</url>
  </repository>

  <repository>
    <id>clojars.org</id>
    <url>http://clojars.org/repo</url>
  </repository>

  <repository>
    <id>twitter4j</id>
    <url>http://twitter4j.org/maven2</url>
  </repository>
</repositories>
```

所有命令行输入和输出如下所示：

```
mkdir FirstGitProject
cd FirstGitProject
git init
```

## 下载本书的示例代码

访问 <http://www.packtpub.com/>，可以下载本书及你所购买的所有 Packt 图书的示例代码。如果你是从其他地方购买的本书英文版，可以访问 <http://www.packtpub.com/support> 并

注册，以便通过电子邮件取得示例文件。

作者通过 Bitbucket 账户来维护开源版本的代码：<https://bitbucket.org/qanderson>。

## 致谢

感谢 Storm 社区，正是他们的努力构建出了开源社区中如此出色的平台，当然，还要特别感谢 Storm 的核心作者——Nathan Marz。

我要感谢我的妻子和孩子对我长时间写作本书和其他相关项目的宽容，感谢这段时间他们给予我的帮助，我爱他们。此外，我还要感谢所有参与本书审校工作的朋友。



# 目 录

译者序

前言

## 第 1 章 搭建开发环境 / 1

- 1.1 简介 / 1
- 1.2 搭建开发环境 / 1
- 1.3 分布式版本控制 / 3
- 1.4 创建“Hello World”Topology / 6
- 1.5 创建 Storm 集群——配置机器 / 12
- 1.6 创建 Storm 集群——配置 Storm / 18
- 1.7 获取基本的点击率统计信息 / 23
- 1.8 对 Bolt 进行单元测试 / 31
- 1.9 实现集成测试 / 34
- 1.10 将产品部署到集群 / 37

## 第 2 章 日志流处理 / 38

- 2.1 简介 / 38
- 2.2 创建日志代理 / 38
- 2.3 创建日志 Spout / 40
- 2.4 基于规则的日志流分析 / 45
- 2.5 索引与持久化日志数据 / 49
- 2.6 统计与持久化日志统计信息 / 53
- 2.7 为日志流集群创建集成测试 / 55
- 2.8 创建日志分析面板 / 59

## 第 3 章 使用 Trident 计算单词重要度 / 71

- 3.1 简介 / 71
- 3.2 使用 Twitter 过滤器创建 URL 流 / 71
- 3.3 从文件中获取整洁的词流 / 76
- 3.4 计算每个单词的相对重要度 / 81

## 第 4 章 分布式远程过程调用 / 85

- 4.1 简介 / 85
- 4.2 通过 DPRC 实现所需处理流程 / 85
- 4.3 对 Trident Topology 进行集成测试 / 90
- 4.4 实现滚动窗口 Topology / 95
- 4.5 在集成测试中模拟时间 / 98

## 第 5 章 在不同语言中实现 Topology / 100

- 5.1 简介 / 100
- 5.2 在 Qt 中实现多语言协议 / 100
- 5.3 在 Qt 中实现 SplitSentence Bolt / 105
- 5.4 在 Ruby 中实现计数 Bolt / 108
- 5.5 在 Clojure 中实现单词计数 Topology / 109

## 第 6 章 Storm 与 Hadoop 集成 / 113

- 6.1 简介 / 113
- 6.2 在 Hadoop 中实现 TF-IDF 算法 / 115
- 6.3 持久化来自 Storm 的文件 / 121
- 6.4 集成批处理与实时视图 / 122

## 第 7 章 实时机器学习 / 127

- 7.1 简介 / 127
- 7.2 实现事务性 Topology / 129
- 7.3 在 R 中创建随机森林分类模型 / 134
- 7.4 基于随机森林的事务流业务分类 / 143

7.5 在 R 中创建关联规则模型 / 149

7.6 创建推荐引擎 / 152

7.7 实时在线机器学习 / 157

## 第 8 章 持续交付 / 162

8.1 简介 / 162

8.2 搭建 CI 服务器 / 162

8.3 搭建系统环境 / 164

8.4 定义交付流水线 / 166

8.5 实现自动化验收测试 / 170

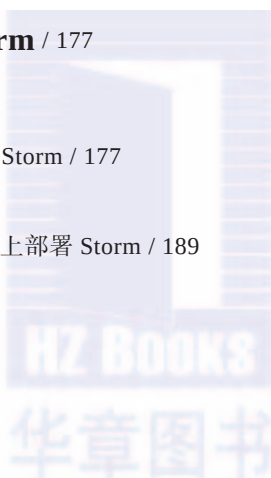
## 第 9 章 在 AWS 上部署 Storm / 177

9.1 简介 / 177

9.2 使用 Pallet 在 AWS 上部署 Storm / 177

9.3 搭建虚拟私有云 / 181

9.4 使用 Vagrant 在虚拟私有云上部署 Storm / 189



# 第 1 章 搭建开发环境

## 1.1 简介

本章将简要介绍 Storm 处理系统。这将涵盖所有你想知道的内容，从搭建你的开发环境到部署 Topology 时需要注意的操作关注事项，再到基本的质量实践，比如对 Storm Topology 进行单元测试和集成测试。在阅读完本章后，你将能够构建、测试和交付基本的 Storm Topology。

本书并不准备对 Storm 处理系统及其元语和架构进行理论介绍。我们假定你在阅读本书之前已经通过浏览如 Storm wiki 这样的在线资源了解了 Storm 的基本概念。



当系统在产品环境中能持续可靠地产生商业价值时，才能交付系统。为了实现这个目的，在开发 Storm Topology 时，必须始终考虑质量问题和操作注意事项。

## 1.2 搭建开发环境

开发环境涵盖了构建 Storm Topology 所需的各种工具和系统。虽说本书重点关注的是每个有技术侧重点的 Storm 交付，但需要指出的是，对于一个软件开发团队来说，无论使用集中式开发环境还是分布式开发环境，都需要更多的工具和流程来保证高效工作，而且不能仅仅局限于本书所讨论的内容。

无论是为了将来的开发工作，还是为了实现书中的例子，以下几类工具和流程都是快速搭建开发环境必不可少的：

- ❑ SDK
- ❑ 版本控制
- ❑ 构建环境
- ❑ 系统配置工具
- ❑ 集群配置工具

书中描述的配置和安装方法都基于 Ubuntu。不管怎么说，适用于 Ubuntu 的这些方法还是比较容易移植到其他 Linux 发行版中的。如果将这些方法应用于其他发行版时出现任何问题，你可以通过 Storm 邮件列表寻求支持：<https://groups.google.com/forum/#!forum/storm-user>。



环境变量是导致系统可操作性和可用性下降的“罪魁祸首”，如果部署环境与开发环境不同，那么环境变量很可能在这时引起大问题。所以尽可能让开发环境和部署环境保持一致。

### 1.2.1 实战

**Step01** 从 Oracle 网站 (<http://www.oracle.com/technetwork/java/javase/downloads/index.html>) 下载最新版本的 J2SE 6 SDK，并通过以下命令进行安装：

```
chmod 775 jdk-6u35-linux-x64.bin
yes | jdk-6u35-linux-x64.bin
mv jdk1.6.0_35 /opt
ln -s /opt/jdk1.6.0_35/bin/java /usr/bin
ln -s /opt/jdk1.6.0_35/bin/javac /usr/bin
JAVA_HOME=/opt/jdk1.6.0_35
export JAVA_HOME
PATH=$PATH:$JAVA_HOME/bin
export PATH
```

**Step02** 然后安装版本控制系统 Git：

```
sudo apt-get install git
```

**Step03** 接下来，安装构建系统 Maven：

```
sudo apt-get install mvn
```

**Step04** 安装 Puppet、Vagrant 和 VirtualBox，以便于对应用程序和环境进行配置：

```
sudo apt-get install virtualbox puppet vagrant
```

**Step05** 最后，你还需要安装一个集成开发环境 (IDE)：

```
sudo apt-get install eclipse
```



自从 Sun 公司被 Oracle 收购以后，人们就一直在争论该使用哪种 Java SDK。虽说作者理解 OpenJDK 的必要性，但书中的例子都是在 Oracle JDK 下测试通过的。总的来说，OpenJDK 和 Oracle JDK 没有什么区别，只是 Oracle JDK 为了追求稳定而在功能方面相对滞后。

### 1.2.2 解析

对于任何 Java 开发工作来说 JDK 都是必不可少的。GIT 是近几年被广泛采用的开源分布式版本控制系统。稍后我们将简要介绍 GIT。

Maven 是一种广泛使用的“约定优于配置”的构建系统。Maven 包含了许多有用的功能，

比如项目对象模型（POM），POM 能够让我们以有效的方式管理库、依赖和版本。Maven 在互联网上拥有许多二进制仓库可供使用，所以我们可以直接用恰当的方式来维护二进制依赖项，然后打包部署 Topology。

由于 DevOps 和持续交付的不断发展，Puppet 系统已经被广泛用于为 Linux、其他一些操作系统和应用程序提供声明式服务器配置。Puppet 能够对服务器的状态和部署环境进行编程，这一点非常重要，因为这样就可以通过 GIT 版本控制系统管理服务器状态，而且对于服务器的修改都可以安全删除。这么做好处很多，包括可确定的服务器平均恢复时间（MTTR）和审计跟踪，说白了就是让系统更加稳定。这对实现持续交付来说至关重要。

Vagrant 是一个在开发过程中非常有用的工具，它能自动配置 VirtualBox 虚拟机，这对 Storm 处理系统这样的以集群为基础的技术来说尤为重要。为了测试一个集群，你必须构建一个真正的机群或配置多台虚拟机。Vagrant 能准确无误地在本地配置虚拟机。



虚拟机在 IT 运维和系统开发过程中能够大显身手，但需要指出的是，虽然我们都知道在本地部署的虚拟机性能不太理想，但其可用性却完全依赖于可用的内存容量。处理能力不是需要考虑的主要问题，因为现代处理器的利用率普遍严重偏低，尽管处理 Topology 时利用率会高一些。建议你的计算机至少保留 8GB 的内存容量。

## 1.3 分布式版本控制

传统版本控制系统都是集中式的。每个客户端都包含一份从当前版本签出的文件，而当前版本则取决于客户端使用的分支。所有历史版本都会存储在服务器上。这样的做法效果不错，不仅能让团队紧密协作，还能知道其他成员在做什么工作。

但集中式服务器存在一些较为明显的缺点，这让分布式版本控制系统的应用范围越来越广。首先，集中式服务器存在单点故障问题，如果服务器由于某种原因死机或无法访问，开发人员会因此难以继续工作。其次，如果服务器上的数据由于某种原因损坏或丢失，代码库的历史记录也就随之丢失。

基于以上两个原因，开源项目极大地推动了分布式版本控制的发展，但究其主要原因，还是分布式系统能帮助我们实现协作模型。开发人员可以在本地环境下用自己的方式进行工作，然后在方便的时候，一次性或多次将这些改动分布到一个或多个远程代码仓库。

除此之外，还有一个很明显的优势就是，由于每一个客户端都拥有代码仓库的完整镜像，因此会自然存在多个仓库备份。所以，如果任何一个客户端或服务器出现问题，都可以轻松恢复，实现数据还原。

## 实战

本书使用 Git 作为分布式版本控制系统。如果要建立一个代码仓库，你可以克隆已存在的代码仓库，或者通过初始化创建一个新的代码仓库。对于新创建的项目来说，应该通过初始化创建一个新的代码仓库。

**Step01** 通过以下命令创建项目目录：

```
mkdir FirstGitProject
cd FirstGitProject
git init
```

**Step02** 为了确认我们之前的步骤是否有效，需要在代码仓库中添加一些文件：

```
touch README.txt
vim README.txt
```

使用 vim 或任何其他文本编辑器，简单增加一些描述性文字并按下插入键。完成后按下 Esc 键，输入 “:wq”，并按下回车键保存。

**Step03** 在提交之前，我们来看看代码仓库的状态：

```
git status
```

该命令会显示出类似于以下内容的信息：

```
# On branch master
# Initial commit
# Untracked files:
#   README.txt
```

**Step04** 你需要使用以下命令，手动添加所有的文件和目录到 Git 代码仓库中：

```
git add README.txt
```

**Step05** 然后提交：

```
git commit -a
```

**Step06** 这时会打开 vim 编辑器并让你添加注释。



你可以在输入命令的同时，使用 -m 参数指定要提交的描述信息。

如果不把代码仓库存储在远程主机上，你将遇到和集中式存储一样的风险问题。所以你应该把代码仓库存储在远程主机上。www.github.com 和 www.bitbucket.org 是两个不错的免费 Git 托管服务，这两个服务都可以通过加密保证你的知识产权不被公共用户访问。本书使用 bitbucket.org 作为示例。要使用该远程托管服务，只需在你的浏览器上访问该站点并注册一个账户。

注册成功后，点击菜单创建一个新的代码仓库，如图 1-1 所示。



图 1-1 点击菜单创建新代码仓库

填写图 1-2 所示内容来创建代码仓库：

图 1-2 创建代码仓库界面

代码仓库创建完成后，你需要将远程代码仓库添加到你的本地代码仓库中，然后向远程代码仓库提交修改。

```
git remote add origin https://[user]@bitbucket.org/[user]/  
firstgitproject.git  
git push origin master
```

你需要使用刚才注册的用户名替换上面命令行中的 [user]。



稍后我们会介绍克隆代码仓库的方法以及如何通过版本控制进行工作。

## 1.4 创建“Hello World”Topology

“Hello World”Topology 和其他所有的“Hello World”应用程序一样，并没有什么实际用途，其目的在于说明一些基本概念。“Hello World”Topology 结构将演示如何创建一个包含简单的 Spout 和 Bolt 的项目，如何构建项目，并在本地集群模式下运行项目。

### 1.4.1 实战

**Step01** 新建一个项目目录，并初始化你的 Git 代码仓库。

```
mkdir HelloWorld
cd HelloWorld
git init
```

**Step02** 然后，我们需要通过以下命令创建 Maven 项目文件：

```
vim pom.xml
```

**Step03** 使用 vim 或任何其他文本编辑器，为“Hello World”项目添加基本的 XML 标签和项目元数据。

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
http://maven.apache.org/xsd/maven-4.0.0.xsd">

  <modelVersion>4.0.0</modelVersion>

  <groupId>storm.cookbook</groupId>
  <artifactId>hello-world</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <packaging>jar</packaging>

  <name>hello-world</name>
  <url>https://bitbucket.org/[user]/hello-world</url>

  <properties>
    <project.build.sourceEncoding>UTF-8</project.build.
sourceEncoding>
  </properties>

</project>
```

**Step04** 然后，我们需要指定从哪一个 Maven 仓库获取依赖项。在 pom.xml 文件的项目标签内添加以下内容：

```
<repositories>

  <repository>
    <id>github-releases</id>
```

```

    <url>http://oss.sonatype.org/content/repositories
      /github-releases/</url>
  </repository>

  <repository>
    <id>clojars.org</id>
    <url>http://clojars.org/repo</url>
  </repository>

  <repository>
    <id>twitter4j</id>
    <url>http://twitter4j.org/maven2</url>
  </repository>
</repositories>

```



你可以使用 .m2 和 settings.xml 文件替换上面指定的这些仓库，虽然具体方法超出了本书的讨论范围，但对于开发团队来说依赖项管理非常重要，因此这种方法会变得十分有用。

**Step05** 然后在项目标签中添加我们所需的依赖项：

```

<dependencies>
  <dependency>
    <groupId>junit</groupId>
    <artifactId>junit</artifactId>
    <version>3.8.1</version>
    <scope>test</scope>
  </dependency>

  <dependency>
    <groupId>storm</groupId>
    <artifactId>storm</artifactId>
    <version>0.8.1</version>
    <!-- 将 Storm JAR 文件的范围设定成 provided，否则这些依赖项会被打包进你的 JAR -->
    <scope>provided</scope>
  </dependency>

  <dependency>
    <groupId>com.googlecode.json-simple</groupId>
    <artifactId>json-simple</artifactId>
    <version>1.1</version>
  </dependency>
</dependencies>

```

**Step06** 最后，我们需要为 Maven 添加 build 插件定义：

```

<build>
  <plugins>
    <!--
      将 maven-assembly-plugin 绑定到 package 阶段，这样就可以创建一个适用于
      集群部署且不含 Storm 依赖项的 jar 文件
    -->
    <plugin>

```

```

<artifactId>maven-assembly-plugin</artifactId>
<configuration>
  <descriptorRefs>
    <descriptorRef>jar-with-dependencies</descriptorRef>
  </descriptorRefs>
  <archive>
    <manifest>
      <mainClass></mainClass>
    </manifest>
  </archive>
</configuration>
<executions>
  <execution>
    <id>make-assembly</id>
    <phase>package</phase>
    <goals>
      <goal>single</goal>
    </goals>
  </execution>
</executions>
</plugin>
<plugin>
  <groupId>com.theoryinpractise</groupId>
  <artifactId>clojure-maven-plugin</artifactId>
  <version>1.3.8</version>
  <extensions>true</extensions>
  <configuration>
    <sourceDirectories>
      <sourceDirectory>src/clj</sourceDirectory>
    </sourceDirectories>
  </configuration>
  <executions>
    <execution>
      <id>compile</id>
      <phase>compile</phase>
      <goals>
        <goal>compile</goal>
      </goals>
    </execution>
    <execution>
      <id>test</id>
      <phase>test</phase>
      <goals>
        <goal>test</goal>
      </goals>
    </execution>
  </executions>
</plugin>
<plugin>
  <groupId>org.apache.maven.plugins</groupId>
  <artifactId>maven-compiler-plugin</artifactId>
  <configuration>
    <source>1.6</source>
    <target>1.6</target>
  </configuration>

```

```

    </plugin>
  </plugins>
</build>

```

**Step07** 当创建好 POM 文件后，通过 “Esc + : + wq + 回车键” 保存文件，并为 Maven 项目创建所需的目录结构：

```

mkdir src
cd src
mkdir test
mkdir main
cd main
mkdir java

```

**Step08** 然后回到项目根目录，并通过以下命令创建 Eclipse 项目文件：

```
mvn eclipse:eclipse
```



Eclipse 的项目文件类似于 .class 文件，属于生成的工件，在提交到 Git 代码仓库的时候不应该包含此文件，特别是因为它们包含了客户机特定的路径。

**Step09** 打开你的 Eclipse 环境并把刚才生成的项目文件导入到工作区中，如图 1-3 所示。

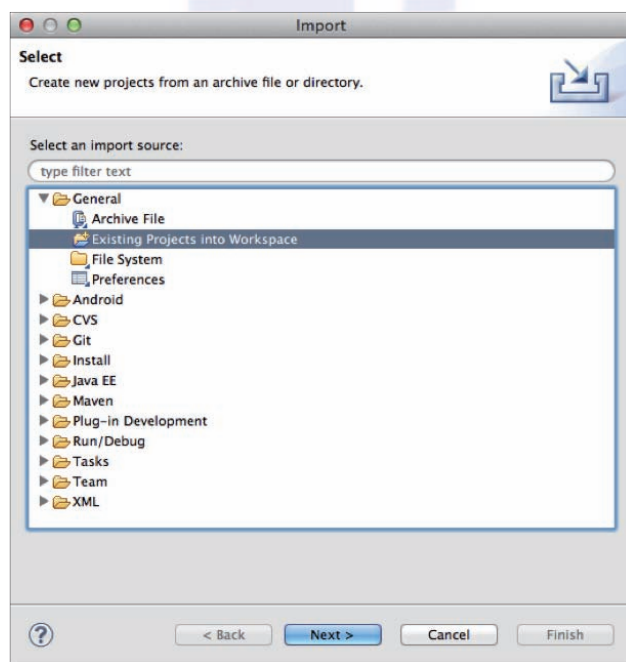


图 1-3 导入项目到工作区

**Step10** 接下来就可以创建你的第一个 Spout 了，在 storm.cookbook 包中创建名为

HelloWorldSpout 的类，并继承 BaseRichSpout。Eclipse 会为你生成一个默认的 spouts 方法。Spout 会生成一个随机数生成的 Tuple。添加以下成员变量并构造对象：

```
private SpoutOutputCollector collector;
private int referenceRandom;
private static final int MAX_RANDOM = 10;
public HelloWorldSpout(){
    final Random rand = new Random();
    referenceRandom = rand.nextInt(MAX_RANDOM);
}
```

**Step11** 对象构造好后，Storm 集群将会打开 Spout，为 open 方法添加以下实现代码：

```
public void open(Map conf, TopologyContext context,
    SpoutOutputCollector collector) {
    this.collector = collector;
}
```

**Step12** Storm 集群会重复调用 nextTuple 方法，该方法包含了整个 Spout 的逻辑。为该方法添加以下实现代码：

```
Utils.sleep(100);
final Random rand = new Random();
int instanceRandom = rand.nextInt(MAX_RANDOM);
if(instanceRandom == referenceRandom){
    collector.emit(new Values("Hello World"));
} else {
    collector.emit(new Values("Other Random Word"));
}
```

**Step13** 最后，你需要通过 declareOutputFields 方法，告诉 Storm 集群 Spout 发送了哪些字段：

```
declarer.declare(new Fields("sentence"));
```

**Step14** 导入类所需的包和类后，就可以开始创建 HelloWorldBolt 了。该类用于读取已产生的 Tuple 并实现必要的统计逻辑。在 storm.cookbook 包中创建一个继承 BaseRichBolt 的新类。为 Bolt 声明一个私有成员变量并为 execute 方法添加以下实现代码：

```
String test = input.getStringByField("sentence");
if("Hello World".equals(test)){
    myCount++;
    System.out.println("Found a Hello World! My Count is now: "
        + Integer.toString(myCount));
}
```

**Step15** 最后，将所有代码逻辑组织起来，声明 Storm Topology。在同一个包中创建名为 HelloWorldTopology 的主类，并添加以下主函数实现：

```
TopologyBuilder builder = new TopologyBuilder();

builder.setSpout("randomHelloWorld", new
    HelloWorldSpout(), 10);
```

```

builder.setBolt("HelloWorldBolt", new
    HelloWorldBolt(), 2)
    .shuffleGrouping("randomHelloWorld");

Config conf = new Config();
conf.setDebug(true);

if(args!=null && args.length > 0) {
    conf.setNumWorkers(3);

    StormSubmitter.submitTopology(args[0], conf,
        builder.createTopology());
} else {

    LocalCluster cluster = new LocalCluster();
    cluster.submitTopology("test", conf,
        builder.createTopology());

    Utils.sleep(10000);
    cluster.killTopology("test");
    cluster.shutdown();
}

```

这样就搭建好了 Topology，并根据传递给 main 方法的参数，将其提交到本地或远程的 Storm 集群上。

**Step16** 解决所有的编译问题后，你就能在项目根目录下，通过以下命令启动集群了：

```

mvn compile exec:java -Dexec.classpathScope=compile -Dexec.
mainClass=storm.cookbook.HelloWorldTopology

```

### 1.4.2 解析

图 1-4 描述了 “Hello World” Topology：

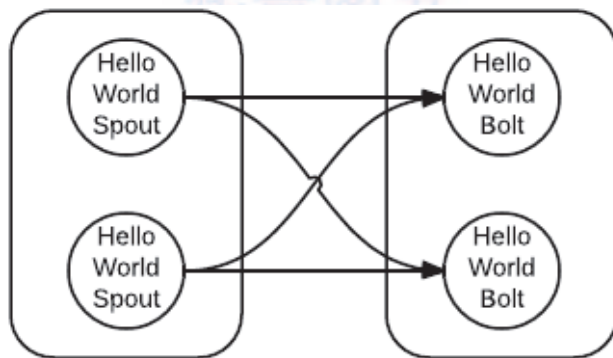


图 1-4 “Hello World” Topology

Spout 最终会发送一个流（Stream），包含下面两条语句中的任意一条：

- ☐ Other Random Word
- ☐ Hello World

Spout 根据随机概率决定要发送哪一条语句，具体原理是：先在构造函数中生成一个随机数，作为原始变量的值，接着生成随后的随机数，并和原始成员变量的值进行比较。若比较结果一致，就发送“Hello World”，否则会发送另一条语句。

Bolt 会简单对比和计算 Hello World 的数量。在我们当前的实现中，你会注意到一系列通过 Bolt 打印出来的增量。想要扩展 Bolt 的规模，只需通过更新以下语句就可以为 Topology 增加并行程度：

```
builder.setBolt("HelloWorldBolt", new HelloWorldBolt(), 3)
    .shuffleGrouping("randomHelloWorld");
```

这里 `parallism_hint` 是关键参数，你可以根据情况把这个参数值调高。如果重新运行集群，你将能看到三条独立的计数信息交织地打印出来。



部署运行后若要调整集群的大小，可以通过 Storm 的 GUI 或 CLI 来修改并行程度。但无法在没有重新编译和部署 JAR 包的情况下，直接修改 Topology 的结构。有关命令行选项的信息，请通过以下链接查看 wiki 上的 CLI 文档：<https://github.com/nathanmarz/storm/wiki/Command-line-client>。

确保 POM 中指定的项目依赖项正确是很重要的。我们必须将 Storm JAR 文件的范围设定成 `provided`，否则这些依赖项会被打包进你的 JAR，这会导致在一个集群节点中，`classpath` 下出现重复的类文件。而且 Storm 会检查这种 `classpath` 重复的问题。启动集群会因为部署中包含了 Storm 的文件而失败。

#### 下载示例代码



访问 <http://www.packtpub.com> 可下载本书及你所购买的所有 Packt 图书的示例代码。如果你是从其他地方购买的本书英文版，可以访问 <http://www.packtpub.com/support> 并注册，以便通过电子邮件取得示例文件。作者通过 Bitbucket 账户来维护开源版本的代码：<https://bitbucket.org/qanderson>。

## 1.5 创建 Storm 集群——配置机器

本地模式下测试集群对调试和验证集群的基本功能逻辑很有帮助。但是，这并不代表你就能够了解集群在实际环境中运行的状况。此外，只有当系统已经在产品环境中运行时，开发工作才算真正完成。任何开发者都应该重视这一点，并且这也是整个 DevOps 实践的基础。无论采用什么方法，你都必须能够将代码可靠地部署到产品环境中。本节将展示如何直接通

过版本控制创建和配置一个完整的集群。在此之前，需要事先说明一些有关创建和配置集群的基本原则：

- ❑ 我们需要时刻了解服务器的会话状态。在没有严格版本控制的情况下，不允许登录和修改服务器的设置或文件。
- ❑ 不能随意修改服务器及其独立存储卷中的状态信息，这样才能确保服务器的恢复时间。
- ❑ 如果在交付过程中遇到了问题，就应该多尝试几次。这对软件开发和信息技术运维中的灾难恢复和集成工作来说，是个不小的负担。而只有恢复和集成工作可以自动化进行的时候，我们才能重复操作。
- ❑ 本书假定你使用的产品部署环境是具有可伸缩性的集群（基于 Amazon Web Services (AWS) EC2）。因为只有具备可伸缩性的集群才能实现自动化配置。

我们会在后续章节介绍 AWS 集群上部署 Storm Topology 的方法，在此之前会在本节的开发环境中对此方法做一些基本的介绍。

### 1.5.1 实战

我们按照下面的方法新建一个项目。

**Step01** 按照以下目录结构，新建一个名为 `vagrant-storm-cluster` 的项目：

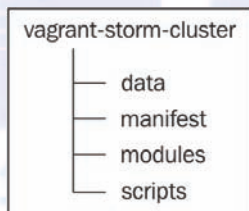


图 1-5 项目的目录结构

**Step02** 用你最喜爱的编辑器，在项目根目录下创建一个名为 `Vagrantfile` 的文件。在文件里添加文件头和我们想创建的虚拟机的配置。文件中至少包含一个 `nimbus` 节点、两个 `supervisor` 节点和一个 `zookeeper` 节点：

```

# -*- mode: ruby -*-
# vi: set ft=ruby :
boxes = [
  { :name => :nimbus, :ip => '192.168.33.100', :cpus => 2, :memory
    => 512 },
  { :name => :supervisor1, :ip => '192.168.33.101', :cpus
    => 4, :memory => 1024 },
  { :name => :supervisor2, :ip => '192.168.33.102', :cpus
    => 4, :memory => 1024 },
  { :name => :zookeeper1, :ip => '192.168.33.201', :cpus
    => 1, :memory => 512 }
]

```



这里要注意的是，由于我们不打算设计具备高可用性的集群，因此在开发环境只用了一个 zookeeper<sup>⊖</sup> 节点。该集群的作用是在现实环境中测试你的 Topology 逻辑，并确定其稳定性。

**Step03** 然后，就可以为每台机器创建虚拟机配置项了，Vagrant 运行的时候会根据这些配置项单独建立每个虚拟机。第一组属性定义了硬件、网络和操作系统：

```
boxes.each do |opts|
  config.vm.define opts[:name] do |config|
    config.vm.box = "ubuntu12"
    config.vm.box_url =
      "http://dl.dropbox.com/u/1537815/precise64.box"
    config.vm.network :hostonly, opts[:ip]
    config.vm.host_name = "storm.%s" % opts[:name].to_s
    config.vm.share_folder "v-data", "/vagrant_data",
      "./data", :transient => false
    config.vm.customize ["modifyvm", :id, "--memory",
      opts[:memory]]
    config.vm.customize ["modifyvm", :id, "--cpus",
      opts[:cpus] ] if opts[:cpus]
```

**Step04** 使用 bash 和 Puppet 混合脚本配置应用程序配置项：

```
config.vm.provision :shell, :inline => "cp -fv /vagrant_data/hosts
/etc/hosts"

config.vm.provision :shell, :inline => "apt-get update"
# 检查 JDK 是否可用
if File.exist?("./data/jdk-6u35-linux-x64.bin") then
  config.vm.provision :puppet do |puppet|
    puppet.manifests_path = "manifests"
    puppet.manifest_file = "jdk.pp"
  end
end

config.vm.provision :puppet do |puppet|
  puppet.manifests_path = "manifests"
  puppet.manifest_file = "provisioningInit.pp"
end

# 用 puppet 进行配置
config.vm.provision :shell, :inline => "puppet apply
/tmp/storm-puppet/manifests/site.pp --verbose --
modulepath=/tmp/storm-puppet/modules/ --debug"

end
end
end
```

Vagrant 文件简单定义了虚拟机管理程序级别的配置项，其余配置项则通过 Puppet 配置，

⊖ zookeeper 是一个针对大型分布式系统的可靠协调服务系统，其高可靠性服务是通过配置多个 zookeeper 节点实现的。——译者注

并定义了两个层次。第一层是安装 Ubuntu，为应用程序配置做准备。第二层则包含了实际的应用程序配置操作。要创建第一层的配置项，需要创建用于 JDK 配置的 bash 脚本和初始化配置项的 Puppet 脚本。

**Step05** 在项目的 scripts 目录中新建 installJdk.sh 文件并添加以下代码：

```
#!/bin/sh
echo "Installing JDK!"
chmod 775 /vagrant_data/jdk-6u35-linux-x64.bin
cd /root
yes | /vagrant_data/jdk-6u35-linux-x64.bin
/bin/mv /root/jdk1.6.0_35 /opt
/bin/rm -rv /usr/bin/java
/bin/rm -rv /usr/bin/javac
/bin/ln -s /opt/jdk1.6.0_35/bin/java /usr/bin
/bin/ln -s /opt/jdk1.6.0_35/bin/javac /usr/bin
JAVA_HOME=/opt/jdk1.6.0_35
export JAVA_HOME
PATH=$PATH:$JAVA_HOME/bin
export PATH
```

我们将会使用声明式的 Puppet 脚本调用该脚本文件。

**Step06** 在 manifest 目录中新建 jdk.pp：

```
$JDK_VERSION = "1.6.0_35"
package {"openjdk":
  ensure => absent,
}
exec { "installJdk":
  command => "installJdk.sh",
  path => "/vagrant/scripts",
  logoutput => true,
  creates => "/opt/jdk${JDK_VERSION}",
}
```

**Step07** 在 manifest 目录中新建 provisioningInit.pp 文件，然后定义需要用到的包和静态变量值：

```
$CLONE_URL = "https://bitbucket.org/qanderson/storm-puppet.git"
$CHECKOUT_DIR="/tmp/storm-puppet"

package {git:ensure=> [latest,installed]}
package {puppet:ensure=> [latest,installed]}
package {ruby:ensure=> [latest,installed]}
package {rubygems:ensure=> [latest,installed]}
package {unzip:ensure=> [latest,installed]}

exec { "install_hiera":
  command => "gem install hiera hiera-puppet",
  path => "/usr/bin",
  require => Package['rubygems'],
}
```



如果需要了解更多有关 Hiera 的信息，请查阅 Puppet 的文档页 <http://docs.puppetlabs.com/hiera/1/index.html>。

**Step08** 然后，我们就可以克隆包含二级配置项的代码仓库了：

```
exec { "clone_storm-puppet":
  command => "git clone ${CLONE_URL}",
  cwd => "/tmp",
  path => "/usr/bin",
  creates => "${CHECKOUT_DIR}",
  require => Package['git'],
}
```

**Step09** 我们还需要配置 Puppet 插件 Hiera，它能以分层方式分离配置脚本的各项属性：

```
exec {"/bin/ln -s /var/lib/gems/1.8/gems/hiera-puppet-1.0.0/ /tmp/
storm-puppet/modules/hiera-puppet":
  creates => "/tmp/storm-puppet/modules/hiera-puppet",
  require => [Exec['clone_storm-
puppet'],Exec['install_hiera']]
}

# 安装 hiera 和 storm 配置文件
file { "/etc/puppet/hiera.yaml":
  source => "/vagrant_data/hiera.yaml",
  replace => true,
  require => Package['puppet']
}

file { "/etc/puppet/hieradata":
  ensure => directory,
  require => Package['puppet']
}

file {"/etc/puppet/hieradata/storm.yaml":
  source => "${CHECKOUT_DIR}/modules/storm.yaml",
  replace => true,
  require => [Exec['clone_storm-puppet'],File['/etc/puppet/
hieradata']]
}
```

**Step10** 最后，还需要在 data 目录中增加几个文件。创建 Hiera 基本配置文件 hiera.yaml：

```
---
:hierarchy:
  - %{operatingsystem}
  - storm
:backends:
  - yaml
:yaml:
  :datadir: '/etc/puppet/hieradata'
```

**Step11** 除此之外，你还需要 host 文件，它在本地集群环境中充当 DNS：

```

127.0.0.1      localhost
192.168.33.100 storm.nimbus
192.168.33.101 storm.supervisor1
192.168.33.102 storm.supervisor2
192.168.33.103 storm.supervisor3
192.168.33.104 storm.supervisor4
192.168.33.105 storm.supervisor5
192.168.33.201 storm.zookeeper1
192.168.33.202 storm.zookeeper2
192.168.33.203 storm.zookeeper3
192.168.33.204 storm.zookeeper4

```



在配置完善的环境中不需要 host 文件，但对于我们这个“只有一个主机”的开发环境来说，host 文件还是非常实用的。

这样就完成了整个项目的构建，虽然它能正确配置虚拟机并安装需要用到的包，但我们还是需要创建 Application 层的配置项，而这些配置项包含在独立的代码仓库中。

**Step12** 为当前项目初始化你的 Git 代码仓库，并将其提交至 bitbucket.org。

## 1.5.2 解析

我们在如图 1-6 所示的三个独立层次上进行配置。

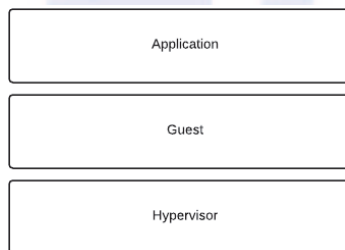


图 1-6 三个独立层次

本节内容只涵盖了配置 Guest 层和 Hypervisor 层的方法，我们会在下一节介绍 Application 层的配置。分层的一个重要原因是，你通常会根据不同的部署管理程序为不同层创建不同的配置项。当配置好虚拟机后，所有环境中的应用栈配置项都应该是一致的。保持配置的一致性非常重要，因为只有在这个条件下，我们才能在完成产品开发前对我们部署的产品进行成百上千次的测试，并保证它们处于可重用和版本控制的状态下。

开发环境中的 VirtualBox 是一个虚拟机管理程序，它通过 Vagrant 和 Puppet 提供其所需的配置项。Vagrant 通过扩展 VirtualBox 镜像实现对虚拟机的配置。VirtualBox 镜像属于受版本控制的工件。Vagrant 文件中定义每个 Box，都需要指定以下参数：

❑ Base Box

- ❑ 网络设置
- ❑ 共享目录
- ❑ 虚拟机的内存和 CPU 设置



这些基本配置项并不包含任何产品环境中你期望的基本控制策略，比如安全、访问控制、托管和监控。所以你必须部署至产品环境中之前配置好这些基本控制策略。你可以在 Puppet Forge 上找到有关内容：<http://forge.puppetlabs.com>。

然后调用配置代理来完成剩余的工作：

```
config.vm.provision :shell, :inline => "cp -fv /vagrant_data/hosts /etc/hosts"
```

执行上面这条命令行语句会安装 host 文件，该文件包含了详细集群信息。

```
config.vm.provision :shell, :inline => "apt-get update"
```

这条语句会更新 Ubuntu apt-get 缓存中所有的包，然后 Vagrant 会安装 JDK 和基本配置项。最后它将调用这些应用程序配置项。



虚拟机镜像可能已经包含了完整的配置，所以不一定要执行以上步骤。但要知道如何创建合适的镜像，以及如何平衡基本镜像中特定配置的数量，否则你的配置数量会过多。

## 1.6 创建 Storm 集群——配置 Storm

当拥有一组准备好进行应用程序配置的虚拟机后，你就可以在每个节点上安装和配置适当的软件包了。

### 1.6.1 实战

**Step01** 按照图 1-7 所示的目录结构，新建名为 storm-puppet 的项目。

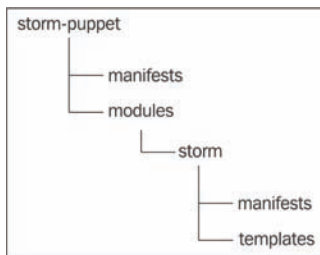


图 1-7 storm-puppet 的目录结构

**Step02** 在配置好的节点上，Puppet 的运行入口（起点）是 `site.pp`。在 `manifests` 目录下新建 `site.pp` 并添加以下内容：

```
node 'storm.nimbus' {
    $cluster = 'storm1'
    include storm::nimbus
    include storm::ui
}

node /storm.supervisor[1-9]/ {
    $cluster = 'storm1'
    include storm::supervisor
}

node /storm.zookeeper[1-9]/ {
    include storm::zoo
}
```

**Step03** 接着，你需要定义 Storm 模块。模块应该放置于 `modules` 目录下，并且和 Puppet 项目根目录的结构一样，拥有自己的 `manifests` 和 `template` 目录结构。在 Storm 模块目录中创建必要的清单文件（`modules/storm/manifests`），首先创建 `init.pp`：

```
class storm {
    include storm::install
    include storm::config
}
```

**Step04** 每个 storm 节点上安装 Storm 应用程序的方法都是一样的，只是配置需要通过预先定义好的模板，根据实际情况进行调整。接着，我们来创建 `install.pp` 文件，该文件会指定需要下载和安装的二进制文件：

```
class storm::install {

    $BASE_URL="https://bitbucket.org/qanderson/storm-deb-
        packaging/downloads/"
    $ZMQ_FILE="libzmq0_2.1.7_amd64.deb"
    $JZMQ_FILE="libjzmq_2.1.7_amd64.deb"
    $STORM_FILE="storm_0.8.1_all.deb"

    package { "wget": ensure => latest }

    # 下载所有文件
    exec { "wget_storm":
        command => "/usr/bin/wget ${BASE_URL}${STORM_FILE}" }
    exec { "wget_zmq":
        command => "/usr/bin/wget ${BASE_URL}${ZMQ_FILE}" }
    exec { "wget_jzmq":
        command => "/usr/bin/wget ${BASE_URL}${JZMQ_FILE}" }

    # 安装所有文件
    package { "libzmq0":
        provider => dpkg,
        ensure => installed,
```

```

source => "${ZMQ_FILE}",
require => Exec['wget_zmq']
}
#call package for each file
package { "libjzmq":
  provider => dpkg,
  ensure => installed,
  source => "${JZMQ_FILE}",
  require => [Exec['wget_jzmq'],Package['libzmq0']]
}
#call package for each file
package { "storm":
  provider => dpkg,
  ensure => installed,
  source => "${STORM_FILE}",
  require => [Exec['wget_storm'], Package['libjzmq']]
}
}

```



这里的 install 清单文件是建立在 Ubuntu 的 Debian 安装包基础之上的。它基于脚本构建，并可以根据需求进行调整。二进制文件和创建脚本可以在 <https://bitbucket.org/qanderson/storm-deb-packaging> 找到。

将会安装以下包：

- ❑ Storm
- ❑ ZeroMQ: <http://www.zeromq.org/>
- ❑ Java-ZeroMQ

**Step05** 每个节点的配置是通过基于模板产生的配置文件生成的。在 storm 清单目录下创建 config.pp:

```

class storm::config {
  require storm::install
  include storm::params
  file { ['/etc/storm/storm.yaml']:
    require => Package['storm'],
    content => template('storm/storm.yaml.erb'),
    owner   => 'root',
    group   => 'root',
    mode    => '0644'
  }
  file { ['/etc/default/storm']:
    require => Package['storm'],
    content => template('storm/default.erb'),
    owner   => 'root',
    group   => 'root',
    mode    => '0644'
  }
}

```

**Step06** 所有 storm 参数都通过 Hiera 定义。Hiera 则会使用 storm 清单目录下 params.pp 定

义的 Hiera 配置：

```
class storm::params {
  #_ STORM DEFAULTS _#
  $java_library_path = hiera_array('java_library_path',
    ['/usr/local/lib', '/opt/local/lib', '/usr/lib'])
}
```



由于属性的数量过多，因此在此省略了配置的其他内容。请到位于 <https://bitbucket.org/ganderson/storm-puppet/src> 的 Git 代码仓库获取完整的清单文件。

**Step07** 然后指定每个节点类。我们先定义 nimbus 类：

```
class storm::nimbus {
  require storm::install
  include storm::config
  include storm::params

  # 安装 nimbus 的 /etc/default 文件
  storm::service { 'nimbus':
    start      => 'yes',
    jvm_memory => $storm::params::nimbus_mem
  }
}
```

然后是 supervisor 类：

```
class storm::supervisor {
  require storm::install
  include storm::config
  include storm::params

  # 安装 supervisor 的 /etc/default 文件
  storm::service { 'supervisor':
    start      => 'yes',
    jvm_memory => $storm::params::supervisor_mem
  }
}
```

接着是 ui 类：

```
class storm::ui {
  require storm::install
  include storm::config
  include storm::params

  # 安装 supervisor 的 /etc/default 文件
  storm::service { 'ui':
    start      => 'yes',
    jvm_memory => $storm::params::ui_mem
  }
}
```

最后指定 zoo 类（供 zookeeper 节点使用）：

```

class storm::zoo {
  package {['zookeeper', 'zookeeper-bin', 'zookeeperd']:
    ensure => latest,
  }
}

```

**Step08** 创建好所有文件后，初始化 Git 代码仓库并提交至 bitbucket.org。

**Step09** 为了确保配置生效，我们还要回到 vagrant-storm-cluster 目录并执行以下命令：

```
vagrant up
```

**Step10** 如果你想通过 ssh 访问任意一个节点，只需执行下面的命令：

```
vagrant ssh nimbus
```

将 nimbus 替换为你所需的节点名。

### 1.6.2 解析

在使用 Puppet 过程中，可以应用多种不同的模式。最简单的方法是使用分布式模型，在这种模式下，每一个节点进行自主配置，这与使用 Puppet Master 的集中式模式完全不同。在分布式模型中，若要更新服务器配置，只需更新配置清单，并将配置清单提交到集中式 Git 代码仓库即可。所有节点随后会获取并应用这些配置。可以通过 cron jobs、触发器，或者使用如 Jenkins、Bamboo、Go 这样的持续集成工具完成这项工作。使用以下命令调用 Vagrant，在开发环境中进行配置：

```

config.vm.provision :shell, :inline => "puppet apply /tmp/storm-puppet/manifests/site.pp --verbose --modulepath=/tmp/storm-puppet/modules/--debug"

```

这时 Puppet 就会应用清单文件。Puppet 是一个使用声明式语言的配置管理工具，其中每一个语言元素都定义了期望的状态以及获得该状态的方法。也就是说，当系统已经处于所需状态下时，Puppet 就会跳过特定的配置步骤，也就避免了重复配置的不利影响。

每个节点都会克隆 storm-puppet 项目，并在其本地应用清单配置。每个节点只应用与自己对应的配置项，而应用哪一项配置取决于 site.pp 清单中定义的主机名，比如：

```

node 'storm.nimbus' {
  $cluster = 'storm1'
  include storm::nimbus
  include storm::ui
}

```

在这种情况下，nimbus 节点将包含 cluster1 的 Hiera 配置，然后将会安装 nimbus 和 ui 节点。由于在 node 定义中可以包含任意类的组合，因此你能够方便地定义整个环境。

## 1.7 获取基本的点击率统计信息

统计点击率 Topology 用于记录基本的网站使用量统计信息，具体来说它会统计以下信息：

- ☐ 访客数量
- ☐ 独立访客数量
- ☐ 特定国家的访客数量
- ☐ 特定城市的访客数量
- ☐ 特定国家中每个城市访客数量的百分比

该系统假定访客总量一定，而且希望服务端的用户标示与客户端的 Cookies 是相对应的。该 Topology 通过 IP 地址和公用 IP 解析服务获取地理位置信息。

统计点击率 Topology 还将使用 Redis 存储发送到 Topology 的点击事件，具体来说就是将其作为一个持久化队列；除此以外还要利用 Redis 记录再次访问网站的访客信息。



关于 Redis 的更多信息，请访问 [Redis.io](http://Redis.io)。

### 1.7.1 准备工作

在我们开始之前，先要安装 Redis（2.6 或更高版本）：

```
wget http://download.redis.io/redis-stable.tar.gz
tar xvzf redis-stable.tar.gz
cd redis-stable
make
sudo cp redis-server /usr/local/bin/
sudo cp redis-cli /usr/local/bin/
```

然后启动 Redis 服务器。

### 1.7.2 实战

**Step01** 新建一个名为 click-topology 的 Java 项目，然后按照创建“Hello World”Topology 项目的方法建立 pom.xml 文件和目录结构。

**Step02** 在 pom.xml 文件中修改对应的项目名称和引用，然后在 <dependencies> 标签中添加以下依赖项：

```
<dependency>
  <groupId>junit</groupId>
  <artifactId>junit</artifactId>
  <version>4.11</version>
  <scope>test</scope>
```

```

</dependency>
<dependency>
  <groupId>org.jmock</groupId>
  <artifactId>jmock-junit4</artifactId>
  <version>2.5.1</version>
  <scope>test</scope>
</dependency>
<dependency>
  <groupId>org.jmock</groupId>
  <artifactId>jmock-legacy</artifactId>
  <version>2.5.1</version>
  <scope>test</scope>
</dependency>
<dependency>
  <groupId>redis.clients</groupId>
  <artifactId>jedis</artifactId>
  <version>2.1.0</version>
</dependency>

```

**Step03** 要特别注意 JUnit 和 JMock 的 scope 定义，避免将它们包含在用于最终部署的 JAR 包中。

**Step04** 在 src/main/java 目录下的 package storm.cookbook 包中创建 ClickTopology 主类。该类定义了 Topology，并提供了在集群或者本地模式下运行 Topology 的机制。按照下面的方式创建这个类：

```

public ClickTopology(){
    builder.setSpout("clickSpout", new ClickSpout(), 10);

    // Bolt 的第一层实现
    builder.setBolt("repeatsBolt", new RepeatVisitBolt(), 10)
        .shuffleGrouping("clickSpout");
    builder.setBolt("geographyBolt", new GeographyBolt(new
        HttpIPResolver()), 10)
        .shuffleGrouping("clickSpout");

    // Bolt 的第二层实现，自然交换
    builder.setBolt("totalStats", new VisitStatsBolt(),
        1).globalGrouping("repeatsBolt");
    builder.setBolt("geoStats", new GeoStatsBolt(),
        10).fieldsGrouping("geographyBolt", new
        Fields(storm.cookbook.Fields.COUNTRY));
    conf.put(Conf.REDIS_PORT_KEY, DEFAULT_JEDIS_PORT);
}

public void runLocal(int runTime){
    conf.setDebug(true);
    conf.put(Conf.REDIS_HOST_KEY, "localhost");
    cluster = new LocalCluster();
    cluster.submitTopology("test", conf,
        builder.createTopology());

    if(runTime > 0){
        Utils.sleep(runTime);
    }
}

```

```

        shutDownLocal();
    }
}

public void shutDownLocal(){
    if(cluster != null){
        cluster.killTopology("test");
        cluster.shutdown();
    }
}

public void runCluster(String name, String redisHost)
    throws AlreadyAliveException,
    InvalidTopologyException {
    conf.setNumWorkers(20);
    conf.put(Config.REDIS_HOST_KEY, redisHost);
    StormSubmitter.submitTopology(name, conf,
        builder.createTopology());
}

```

**Step05** 然后实现 main 方法，该方法根据运行时传递进来的参数个数来决定使用哪种 Topology 运行模式：

```

public static void main(String[] args) throws Exception {
    ClickTopology topology = new ClickTopology();

    if(args!=null && args.length > 1) {
        topology.runCluster(args[0], args[1]);
    } else {
        if(args!=null && args.length == 1)
            System.out.println("Running in local mode,
                redis ip missing for cluster run");
        topology.runLocal(10000);
    }
}

```

**Step06** 这个 Topology 假定由 Web 服务器向 Redis 队列推送消息。因此你必须创建一个 Spout，才能将这些消息作为一个流添加到 Storm 集群中。在 storm.coobook 包中创建 ClickSpout 类，用于在集群启动时建立与 Redis 的连接：

```

public class ClickSpout extends BaseRichSpout {

    public static Logger LOG =
        Logger.getLogger(ClickSpout.class);

    private Jedis jedis;
    private String host;
    private int port;
}

```

```

private SpoutOutputCollector collector;

@Override
public void declareOutputFields(OutputFieldsDeclarer
    outputFieldsDeclarer) {
    outputFieldsDeclarer.declare(new
        Fields(storm.cookbook.Fields.IP,
            storm.cookbook.Fields.URL,
            storm.cookbook.Fields.CLIENT_KEY));
}

@Override
public void open(Map conf, TopologyContext
    topologyContext, SpoutOutputCollector
    spoutOutputCollector) {
    host = conf.get(Conf.REDIS_HOST_KEY).toString();
    port = Integer.valueOf(conf
        .get(Conf.REDIS_PORT_KEY).toString());
    this.collector = spoutOutputCollector;
    connectToRedis();
}

private void connectToRedis() {
    jedis = new Jedis(host, port);
}

```

**Step07** 集群会通过 nextTuple 方法从 Spout 获取新的 Tuple:

```

public void nextTuple() {
    String content = jedis.rpop("count");
    if(content==null || "nil".equals(content)) {
        try { Thread.sleep(300); }
        catch (InterruptedException e) {}
    } else {
        JSONObject obj=(JSONObject) JSONValue.parse(content);
        String ip = obj.get(storm.cookbook.Fields.IP).toString();
        String url = obj.get(storm.cookbook.Fields.URL).toString();
        String clientKey = obj.get(storm.cookbook.Fields.CLIENT_KEY)
            .toString();
        collector.emit(new Values(ip,url,clientKey));
    }
}

```

**Step08** 接下来，我们需要创建 Bolt，其作用是通过数据库或者远程 API 查询来丰富基本数据。让我们从“处理重复访客信息 Bolt”开始创建，该 Bolt 会比较客户端 ID 与之前访问记录是否一致，以此判断用户是否第一次访问，然后发送处理后包含这个标记的 Tuple。创建 RepeatVisitBolt 类，提供打开逻辑和 Redis 连接逻辑：

```

public class RepeatVisitBolt extends BaseRichBolt {

    private OutputCollector collector;

    private Jedis jedis;
    private String host;
    private int port;

    @Override
    public void prepare(Map conf,
        TopologyContext topologyContext, OutputCollector
        outputCollector) {
        this.collector = outputCollector;
        host = conf.get(Conf.REDIS_HOST_KEY).toString();
        port = Integer.valueOf(conf.
            get(Conf.REDIS_PORT_KEY).toString());
        connectToRedis();
    }

    private void connectToRedis() {
        jedis = new Jedis(host, port);
        jedis.connect();
    }
}

```

**Step09** 在 `execute` 方法中，`Tuple` 由集群中的 `ClickSpout` 类提交。Bolt 会基于 `Tuple` 里的字段，从 Redis 查询之前的访问标记，然后发送处理后的 `Tuple`：

```

public void execute(Tuple tuple) {
    String ip = tuple.getStringByField(storm.cookbook.Fields.IP);
    String clientKey = tuple.getStringByField(storm.cookbook.Fields
        .CLIENT_KEY);

    String url = tuple.getStringByField(storm.cookbook.Fields.URL);
    String key = url + ":" + clientKey;
    String value = jedis.get(key);
    if(value == null){
        jedis.set(key, "visited");
        collector.emit(new Values(clientKey, url,
            Boolean.TRUE.toString()));
    } else {
        collector.emit(new Values(clientKey, url,
            Boolean.FALSE.toString()));
    }
}
}

```

**Step10** 接下来就该创建“处理地理位置信息 Bolt”了。该 Bolt 会通过远程 API 调用查询客户 IP 地址所对应的国家和城市。GeographyBolt 类把实际的调用委托给一个预先添加的 IP 解析器，以便于提高类的可测试性。在 `storm.cookbook` 包中创建 `GeographyBolt` 类，并继承 `BaseRichBolt` 接口，然后实现 `execute` 方法：

```

public void execute(Tuple tuple) {
    String ip = tuple.getStringByField(storm
        .cookbook.Fields.IP);
    JSONObject json = resolver.resolveIP(ip);
    String city = (String) json.get(storm
        .cookbook.Fields.CITY);
    String country = (String) json.get(storm
        .cookbook.Fields.COUNTRY_NAME);
    collector.emit(new Values(country, city));
}

```

**Step11** 为了能够使用这个解析器，我们需要自己实现一个名为 `HttpIPResolver` 的解析器，并在设计的时候就将其添加到 `GeographyBolt` 类的实现当中：

```

public class HttpIPResolver implements IPResolver, Serializable {
    static String url =
        "http://api.hostip.info/get_json.php";
    @Override
    public JSONObject resolveIP(String ip) {
        URL geoUrl = null;
        BufferedReader in = null;
        try {
            geoUrl = new URL(url + "?ip=" + ip);
            URLConnection connection = geoUrl.openConnection();
            in = new BufferedReader(new InputStreamReader(
                connection.getInputStream()));
            JSONObject json = (JSONObject) JSONValue.parse(in);
            in.close();
            return json;
        } catch (IOException e) {
            e.printStackTrace();
        }
        finally {
            if(in != null){
                try {
                    in.close();
                } catch (IOException e) {}
            }
        }
        return null;
    }
}

```

**Step12** 接下来，我们还需要获取地理位置统计信息。`GeoStatsBolt` 类的原理很简单，它从 `GeographicBolt` 类接收处理完成后的 `Tuple`，与此同时在内存中维护这个数据结构。它还可以发送更新的统计信息给订阅者。`GeoStatsBolt` 类的设计初衷是让我们能够把不同国家的总人口数据分割到不同的 `Bolt` 中进行处理。但需要确保一个国家所有城市的数据在同一个 `Bolt` 中进行处理。`Topology` 就是根据不同国家这一点将流分割并添加到 `Bolt` 中的。

```
builder.setBolt("geoStats", new GeoStatsBolt(),
    10).fieldsGrouping("geographyBolt", new
    Fields(storm.cookbook.Fields.COUNTRY));
```

**Step13** 创建 GeoStatsBolt 类，并实现 execute 方法：

```
public void execute(Tuple tuple) {
    String country = tuple.getStringByField(storm.cookbook.
    Fields.COUNTRY);
    String city = tuple.getStringByField(Fields.CITY);
    if(!stats.containsKey(country)){
        stats.put(country, new CountryStats(country));
    }
    stats.get(country).cityFound(city);
    collector.emit(new Values(country,
        stats.get(country).getCountryTotal(), city,
        stats.get(country).getCityTotal(city)));
}
```

**Step14** 大部分逻辑都包含在内部模型类中，这个类在内存中维护城市和国家信息：

```
private class CountryStats {
    private int countryTotal = 0;
    private static final int COUNT_INDEX = 0;
    private static final int PERCENTAGE_INDEX = 1;
    private String countryName;
    public CountryStats(String countryName){
        this.countryName = countryName;
    }
    private Map<String, List<Integer>> cityStats = new
        HashMap<String, List<Integer>>();
    public void cityFound(String cityName){
        countryTotal++;
        if(cityStats.containsKey(cityName)){
            cityStats.get(cityName).set(COUNT_INDEX,
                cityStats.get(cityName)
                    .get(COUNT_INDEX).intValue() + 1);
        } else {
            List<Integer> list = new
                LinkedList<Integer>();
            // 添加一些假的数据
            list.add(1);
            list.add(0);
            cityStats.put(cityName, list);
        }

        double percent = (double)cityStats.get(cityName)
            .get(COUNT_INDEX)/(double)countryTotal;
        cityStats.get(cityName).set(PERCENTAGE_INDEX,
            (int)percent);
    }
}
```

```

    public int getCountryTotal(){return countryTotal;}
    public int getCityTotal(String cityName){
        return cityStats.get(cityName)
            .get(COUNT_INDEX).intValue();
    }
}

```

**Step15** 最后，基于 RepeatVisitBolt 类处理完成后的流，VisitorStatsBolt 方法对访客数量和独立访客数量进行最终统计。这个 Bolt 在内存中维护了访客与独立访客的数量，它需要接收所有的计数信息并进行统计，我们能通过 Topology 中的定义看出些端倪：

```

builder.setBolt("totalStats", new VisitStatsBolt(), 1).globalGrouping("repeatsBolt");

```

**Step16** 现在来实现 VisitorStatsBolt 类，创建这个类并定义两个整型成员变量，total 和 uniqueCount，然后实现 execute 方法：

```

public void execute(Tuple tuple) {
    boolean unique = Boolean.parseBoolean(tuple
        .getStringByField(storm.cookbook.Fields.UNIQUE));
    total++;
    if(unique)uniqueCount++;
    collector.emit(new Values(total,uniqueCount));
}

```

### 1.7.3 解析

图 1-8 描述了统计点击率 Topology。

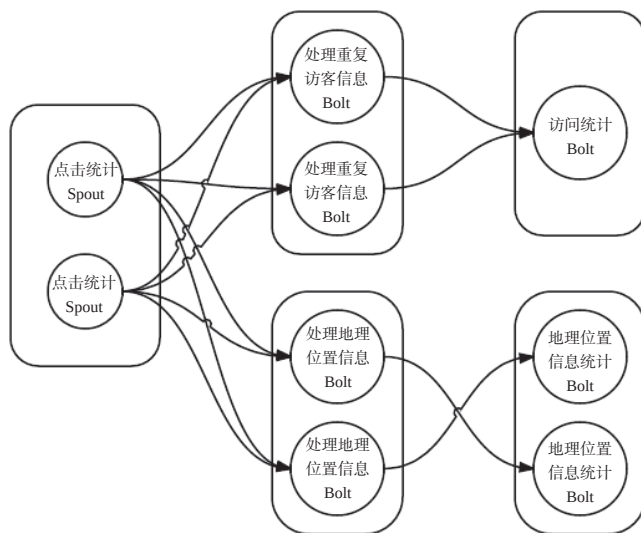


图 1-8 统计点击率 Topology

Spout 把 Web 服务器上获得的点击事件发送到 Topology 中，再通过随机分组转发给“处理地理位置信息 Bolt”和“处理重复访客信息 Bolt”。这样能保证集群内的负载均衡，对于像这样处理速度慢或高延时的处理过程来说负载均衡尤为重要。



在设计 Topology 之前，需要充分理解数据模型的交换结合性质、你的流的其他问题和结构的固有模型。

在搭建 Topology 的时候，要对 Storm 的并行性有充分的认识。Storm wiki 上有一篇文章对这个问题作了很好的总结，网址是 <https://github.com/nathanmarz/storm/wiki/Understanding-the-parallelism-of-a-Storm-topology>。其中有几个要点需要注意：

- ❑ Topology 中工作进程 (Worker) 的数量 (TOPOLOGY\_WORKERS)。
- ❑ Topology 中每个组件执行单元 (线程) 的数量。这个值取决于并行程度。值得注意的是，这组值只是初始值 (线程数)，运行过程中可通过 Topology 重新配置 (通过 UI 或 CLI) 执行单元的数量。还可以通过 `Config#setMaxTaskParallelism()` 方法限制执行单元的数量。
- ❑ 每个执行单元的默认任务数都是 1。可以在声明新组件的时候通过 `ComponentConfigurationDeclarer#setNumTasks()` 方法调整这个值。

这些是调整集群大小时要点。集群会尝试分发任务给工作进程，每个进程包含许多执行单元，而每个执行单元又可能在运行一个或多个任务。每个工作进程的执行单元数量其实是一个执行单元总数量和执行进程总数量之比的函数。关于这个问题，可以在刚才提到的 wiki 页面上找到一个很好的例子。

可以参考节点的数量和每个节点处理器的核数来调整集群大小。我们推荐为每个线程 (执行单元) 分配一个核。

## 1.8 对 Bolt 进行单元测试

对于任何产品交付来说，单元测试都是其重要组成部分，我们也必须对 Bolt 中实现的逻辑进行单元测试。

### 1.8.1 准备工作

单元测试中有一种很常用的技术叫做 Mock，它允许你动态生成依赖对象的假实例，这样才能保证对一个特定类进行准确的单元测试。本书通过 JUnit 4 和 JMock 来演示单元测试。请花些时间阅读 JMock 使用方法，网址是 <http://jmock.org/cookbook.html>。

## 1.8.2 实战

**Step01** 在 src/test/java 目录下创建 storm.cookbook 包，然后创建 StormTestCase 类。该类是一些初始化代码的抽象类：

```
public class StormTestCase {

    protected Mockery context = new Mockery() {{
        setImposteriser(ClassImposteriser.INSTANCE);
    }};

    protected Tuple getTuple(){
        final Tuple tuple = context.mock(Tuple.class);
        return tuple;
    }
}
```

**Step02** 创建 TestRepeatVisitBolt 类，并继承 StormTestCase，然后使用参数化执行器注解来标记它：

```
@RunWith(value = Parameterized.class)
public class TestRepeatVisitBolt extends StormTestCase {
```

**Step03** execute 方法中涵盖了类的测试用例逻辑：

```
public void testExecute(){
    jedis = new Jedis("localhost", 6379);
    RepeatVisitBolt bolt = new RepeatVisitBolt();
    Map config = new HashMap();
    config.put("redis-host", "localhost");
    config.put("redis-port", "6379");
    final OutputCollector collector =
        context.mock(OutputCollector.class);
    bolt.prepare(config, null, collector);

    assertEquals(true, bolt.isConnected());

    final Tuple tuple = getTuple();
    context.checking(new Expectations(){{
        oneOf(tuple).getStringByField(Fields
            .IP);will(returnValue(ip));
        oneOf(tuple).getStringByField(Fields
            .CLIENT_KEY);will(returnValue(clientKey));
        oneOf(tuple).getStringByField(Fields
            .URL);will(returnValue(url));
        oneOf(collector).emit(new Values
            (clientKey, url, expected));
    }});
}
```

```

    bolt.execute(tuple);
    context.assertIsSatisfied();

    if(jedis != null)
        jedis.disconnect();
}

```

**Step04** 接下来，我们还需要定义一些参数：

```

@Parameterized.Parameters
public static Collection<Object[]> data() {
    Object[][] data = new Object[][] {
        { "192.168.33.100", "Client1", "myintranet.com", "false" },
        { "192.168.33.100", "Client1", "myintranet.com", "false" },
        { "192.168.33.101", "Client2", "myintranet1.com", "true" },
        { "192.168.33.102", "Client3", "myintranet2.com", "false" };
    return Arrays.asList(data);
}

```

**Step05** 在使用 Redis 进行测试之前，必须初始化一些基本配置的值：

```

@BeforeClass
public static void setupJedis(){
    Jedis jedis = new Jedis("localhost",6379);
    jedis.flushDB();
    Iterator<Object[]> it = data().iterator();
    while(it.hasNext()){
        Object[] values = it.next();
        if(values[3].equals("false")){
            String key = values[2] + ":" + values[1];
            jedis.set(key, "visited");//unique, meaning
                                   it must exist
        }
    }
}

```



保留测试完成后栈中的数据总是有用的，这样便于复查和调试逻辑。我们只要在下一次测试执行时清理栈中的数据就好了。

### 1.8.3 解析

首先，单元测试是由一组测试数据驱动的测试。这就是说我们能在测试不同用例的时候免去许多无谓和重复的操作。在测试运行之前会将静态数据写入 Redis 数据库中，保证了测试的正确运行。接着测试程序就会一行一行地执行参数化的测试数据，并验证许多不同的用例。

JMock 负责 Mock 收集器及 Bolt 发送的 Tuple。然后通过 Mock 的对象与对象之间的交互来定义我们预期执行的操作：

```
context.checking(new Expectations(){{
    oneOf(tuple).getStringByField(Fields.IP);will(returnValue(ip));
    oneOf(tuple).getStringByField(Fields.CLIENT_KEY);will(returnValue(
        clientKey));
    oneOf(tuple).getStringByField(Fields.URL);will(returnValue(url));
    oneOf(collector).emit(new Values(clientKey, url, expected));
}});
```

我们应该把 Expectations 对象内的代码当做描述性的语言来理解。比如方法体第一行的意思就是：希望 Tuple 的 getStringByField 方法被精确地调用一次（必须只被调用一次），而且必定会返回一个值给被测试的对象。

这种机制能够让我们准确地测试 Bolt。



还有很多种不同的单元测试方法，我们通常会利用数据库来实现测试。除非万不得已，还是建议 Mock 所有类的依赖项，这样才能实现真正意义上的单元测试。对于“地理位置信息 Bolt”的 resolver 抽象结构来说，我们完全可以 Mock 所有成员。

## 1.9 实现集成测试

由于环境和使用者的差异，集成测试可能会代表多种不同的概念。在本书中，集成测试指的是根据事先定义的输入和输出点，通过端到端的方法测试本地集群 Topology 的方法。这样就能确保在部署到实际的集群环境之前对 Topology 进行完整的功能性验证。

### 1.9.1 实战

**Step01** 在 src/test/java 目录下的 storm.cookbook 包中创建 IntegrationTestTopology 类。我们在这里添加一个“测试 Bolt”类，以便于搭建本地 Topology：

```
@BeforeClass
public static void setup(){
    // 我们希望所有的输出 Tuple 都会流动到这个测试用例的 mock 对象上
    topology.getBuilder().setBolt("testBolt", testBolt, 1)
        .globalGrouping("geoStats")
        .globalGrouping("totalStats");

    // 在本地模式下运行，不过当任务结束的时候我们会停止集群
    topology.runLocal(0);
    // 集群的输入和输出需要使用 Jedis
    jedis = new Jedis("localhost",
        Integer.parseInt(ClickTopology.DEFAULT_JEDIS_PORT));
    jedis.connect();
```

```

        jedis.flushDB();
// 在开始测试之前等待一会儿，等待任务启动
        Utils.sleep(5000);
    }

```

**Step02** 然后，定义一组成对数组作为所需的参数：

```

@Parameterized.Parameters
    public static Collection<Object[]> data() {
        Object[][] data = new Object[][] { {new Object[]{
            "165.228.250.178", "internal.com", "Client1"}, // 输入数据
            new Object[]{ "AUSTRALIA", new Long(1), "SYDNEY", new Long(1),
                new Long(1), new Long(1) } }, // 期望集
            {new Object[]{ "165.228.250.178", "internal.com",
                "Client1"}, // 输入数据
            new Object[]{ "AUSTRALIA", new Long(2), "SYDNEY", new
                Long(2), new Long(2), new Long(1) } },
            {new Object[]{ "4.17.136.0", "internal.com",
                "Client1"}, // 输入数据，相同客户，不同地点
                different location
            new Object[]{ "UNITED STATES", new Long(1), "DERRY",
                NH", new Long(1), new Long(3), new
                Long(1) } },
            {new Object[]{ "4.17.136.0", "internal.com",
                "Client2"}, // 输入数据，相同客户，不同地点
                different location
            new Object[]{ "UNITED STATES", new Long(2), "DERRY",
                NH", new Long(2), new Long(4), new
                Long(2) } } }; // 期望集

        return Arrays.asList(data);
    }
    Object[] input;
    Object[] expected;
    public IntegrationTestTopology(Object[] input, Object[]
        expected){

        this.input = input;
        this.expected = expected;
    }

```

**Step03** 这样我们就可以根据这些参数编写测试逻辑了：

```

@Test
    public void inputOutputClusterTest(){
        JSONObject content = new JSONObject();
        content.put("ip", input[0]);
        content.put("url", input[1]);
        content.put("clientKey", input[2]);

        jedis.rpush("count", content.toJSONString());

        Utils.sleep(3000);

        int count = 0;
        String data = jedis.rpop("TestTuple");

        while(data != null){

```

```

JSONArray values = (JSONArray)
    JSONValue.parse(data);

if(values.get(0).toString().contains("geoStats")){
    count++;
    assertEquals(expected[0],
        values.get(1).toString().toUpperCase());
    assertEquals(expected[1], values.get(2));
    assertEquals(expected[2],
        values.get(3).toString().toUpperCase());
    assertEquals(expected[3], values.get(4));
} else if(values.get(0).toString().
contains("totalStats")) {
    count++;
    assertEquals(expected[4], values.get(1));
    assertEquals(expected[5], values.get(2));
}
data = jedis.rpop("TestTuple");
}
assertEquals(2, count);
}

```

### 1.9.2 解析

该集成测试首先会创建本地集群，然后通过 Redis 将输入数据添加到集群中，这个过程与实际 Web 服务器的工作原理一致。然后它会把预先定义好的“测试 Bolt”连接到 Topology 结构的末尾，这样我们就能接收所有输出的 Tuple 并与期望值比较。

一旦 TestBolt 的值提交到了集群，在测试过程中就再也不能访问它了。所以只能通过数据持久化手段访问这些输出数据。TestBolt 会把收到的 Tuple 数据持久化存储在 Redis 上，这样测试用例就可以读取并验证这些值了。TestBolt 的代码逻辑如下所示：

```

public void execute(Tuple input) {
    List objects = input.getValues();
    objects.add(0, input.getSourceComponent());
    jedis.rpush("TestTuple", JSONArray.toJSONString(objects));
}

```

然后读取这些值，并与我们所期望的值进行比较：

```

String data = jedis.rpop("TestTuple");

while(data != null){
    JSONArray values = (JSONArray) JSONValue.parse(data);
    if(values.get(0).toString().contains("geoStats")){
        count++;
        assertEquals(expected[0], values.get(1)

```

```

        .toString().toUpperCase());
    assertEquals(expected[1], values.get(2));
    assertEquals(expected[2], values.get(3)
        .toString().toUpperCase());
    assertEquals(expected[3], values.get(4));
} else if(values.get(0).toString().contains("totalStats"))
{
    count++;
    assertEquals(expected[4], values.get(1));
    assertEquals(expected[5], values.get(2));
}
data = jedis.rpop("TestTuple");
}
assertEquals(2, count);
}

```

## 1.10 将产品部署到集群

在完成整个开发过程之前，我们需要对集群中的 Topology 进行功能性测试，然后再将其部署到下个环境中。

### 1.10.1 实战

**Step01** 首先需要在主机开发环境中配置 Storm 客户端，在用户主目录下创建 .storm 目录。然后在这个目录下新建 storm.yaml，内容如下：

```

storm.local.dir: "/mnt/storm"
nimbus.host: "192.168.33.100"

```

**Step02** 在项目根目录下，使用以下命令打包 Topology：

```
mvn package
```

**Step03** 这样就会在项目的目标目录下生成一个完整并打包好的 JAR 文件。这时就可以用 storm 客户端命令将 JAR 包部署到集群上：

```
storm jar jarName.jar [TopologyName] [Args]
```

### 1.10.2 解析

Storm 命令行客户端为你提供了所有控制集群功能的工具。其中一个功能就是部署打包好的 Topology。如果想了解更多有关 storm CLI 的信息，可以阅读 Storm wiki 上提供的详细文档，地址是 <https://github.com/nathanmarz/storm/wiki/Command-line-client>。

## 第 2 章 日志流处理

### 2.1 简介

本章将会展示一个企业日志存储系统的实现方法，以及一个基于 Storm 处理系统的搜索分析解决方案。其实日志数据处理已经不再是一个需要解决的问题了，但它依然能够帮助我们加深理解这些新概念。

数据流处理在现代企业中是一个主要的架构关注点。但通常情况下，最理想的数据流也只是半结构化的。本章展示了一个企业日志的处理方案，目的是为了读者学习了解各种重要概念，获得处理各种类型数据的能力。由于日志数据量庞大，因此日志数据也为学术研究提供了极大的便利。对于任何流处理或分析工作来说，其成功的关键都在于深入理解实际数据和获取数据这两项因素上，而这往往非常困难。

因此，如何将架构设计蓝图运用到企业其他形式的数据中，才是读者需要重点关注的问题。

图 2-1 展示了我们将在本章中介绍的基本概念。

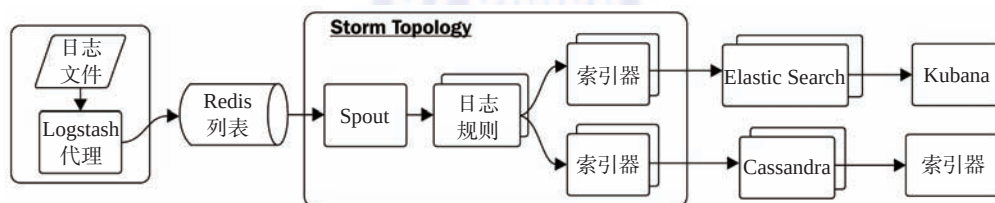


图 2-1 架构设计图

你将学习如何建立日志代理，它可以部署到你环境中的任意节点上。你也将学到如何使用 Storm 和 Redis 集中式地收集、分析、索引和统计日志，以便日后搜索和展示它们的基本统计信息。

### 2.2 创建日志代理

现代企业架构由大量的解决方案组成，而每个方案都包含许多节点。有些 MapReduce 集群可以包含几百个节点。在操作系统和应用层上，每个节点又包含一组应用和服务。这些服务和应用又会产生大量不同类型的日志数据。如今人们已经越来越认识到日志数据在企业团

体中的重要性，原因如下：

- ❑ 对于 IT 运营团队维护系统运行来说，它是一种关键的信息来源。
- ❑ 它对于在生产和系统测试阶段中发现问题和解决问题至关重要。
- ❑ 它越来越成为一种商业价值来源，有价值的业务数据都包含在这种半结构化数据中，包括：
  - ❑ 风险和合规性数据
  - ❑ 业务运营数据
  - ❑ 网站分析数据
  - ❑ 安全信息
  - ❑ 财务预测数据

为了利用这些具有价值的日志数据，我们必须先从这些节点中获取数据，然后再把数据安全简单地传送到中央日志服务器中进行存储、索引和分析。本节将通过名为 logstash 的开源日志代理阐述实现这一过程的方法。



可以找到很多好的商业和开源的日志解决方案。本章只涉及 logstash 的一部分。欲了解更多 logstash 技巧，可以访问 <http://cookbook.logstash.net/> 和 <http://logstash.net/docs/1.1.13/tutorials/getting-started-centralized>。另外还有一个与 logstash 类似的商业版本叫做 Splunk (<http://www.splunk.com/>)。

### 2.2.1 实战

**Step01** 首先，将本地节点上的日志导入 Topology。按照下面的链接下载和配置 logstash：

```
wget https://logstash.objects.dreamhost.com/release/logstash-1.1.7-monolithic.jar
```

**Step02** 然后，使用文本编辑器创建一个名为 shipper.conf 的文件，文件内容如下：

```
input {
  file {
    type => "syslog"

    # 这里可以使用通配符 :)
    path => [ "/var/log/messages", "/var/log/system.*", "/var/log/*.log" ]
  }
}

output {

  # 供调试使用，通过标准输出打印事件。如果不需要可以删除这个输出信息。
  stdout { }
```

```

redis {
  host => "localhost"
  data_type => "list"
  key => "rawLogs"
}
}

```

**Step03** 启动 Redis 本地实例后，执行以下命令启动日志代理：

```
java -jar logstash-1.1.7-monolithic.jar agent-f shipper.conf
```

## 2.2.2 解析

logstash 使用可扩展插件列表实现了一个非常简单的“输入 - 过滤器 - 输出”（input-filter-output）模型，并基于该模型的三种元素不断进行扩展。配置文件（shipper.conf）至少配置了一组输入值和输出值。

文件输入插件根据指定路径中的文件名或通配符来跟踪（tail）文件的修改。你可以配置很多不同类型的文件输入。日志类型在后期处理和分类时尤为重要。由于我们没有在配置文件中配置任何过滤器，因此原始日志将直接被传输到输出插件。基于 Redis 的输出插件将日志输出到本地 Redis 实例中，然后存储到名为 rawLogs 的列表里。



你可以轻松地在设备节点中安装和配置 logstash，包括密钥交换，这样你就可以通过任意传输机制安全地传输日志了。

## 2.3 创建日志 Spout

日志 Topology 通过 Redis 通道读取所有 logstash 产生的日志，这些日志数据会通过本章介绍的 Spout 发送到 Topology 中。由于这是一个全新的 Topology，因此我们先来新建一个 Topology 项目。

### 2.3.1 实战

我们先来创建项目目录和标准的 Maven 目录结构（标准结构可以参考：<http://maven.apache.org/guides/introduction/introduction-to-the-standard-directory-layout.html>）。

**Step01** 参照第 1 章中创建“Hello World”Topology 的方法创建 POM 文件，修改 <artifactId> 和 <name> 标签，然后添加以下依赖项：

```

<dependency>
  <groupId>junit</groupId>
  <artifactId>junit</artifactId>
  <version>4.11</version>

```

```

        <scope>test</scope>
    </dependency>
    <dependency>
        <groupId>org.slf4j</groupId>
        <artifactId>slf4j-log4j12</artifactId>
        <version>1.6.1</version>
    </dependency>
    <dependency>
        <groupId>org.jmock</groupId>
        <artifactId>jmock-legacy</artifactId>
        <version>2.5.1</version>
        <scope>test</scope>
    </dependency>
    <dependency>
        <groupId>storm</groupId>
        <artifactId>storm</artifactId>
        <version>0.8.1</version>
        <!-- 将 Storm JAR 文件的范围设定成 provided, 否则这些依赖项会被打包进你的 JAR -->
        <scope>provided</scope>
        <exclusions>
            <exclusion>
                <artifactId>slf4j-api</artifactId>
                <groupId>org.slf4j</groupId>
            </exclusion>
        </exclusions>
    </dependency>
    <dependency>
        <groupId>com.googlecode.json-simple</groupId>
        <artifactId>json-simple</artifactId>
        <version>1.1</version>
    </dependency>
    <dependency>
        <groupId>redis.clients</groupId>
        <artifactId>jedis</artifactId>
        <version>2.1.0</version>
    </dependency>
    <dependency>
        <groupId>commons-httpclient</groupId>
        <artifactId>commons-httpclient</artifactId>
        <version>3.1</version>
    </dependency>
    <dependency>
        <groupId>org.jmock</groupId>
        <artifactId>jmock-junit4</artifactId>
        <version>2.5.1</version>
        <scope>test</scope>
    </dependency>
    <dependency>
        <groupId>com.github.ptgoetz</groupId>
        <artifactId>storm-cassandra</artifactId>

```

```

        <version>0.3.1-SNAPSHOT</version>
    </dependency>
    <dependency>
        <groupId>org.elasticsearch</groupId>
        <artifactId>elasticsearch</artifactId>
        <version>0.20.2</version>
    </dependency>
    <dependency>
        <groupId>org.drools</groupId>
        <artifactId>drools-core</artifactId>
        <version>5.5.0.Final</version>
    </dependency>
    <dependency>
        <groupId>org.drools</groupId>
        <artifactId>drools-compiler</artifactId>
        <version>5.5.0.Final</version>
    </dependency>
</dependencies>

```

**Step02** 通过以下命令生成 Eclipse 工程文件，然后将工程文件导入 Eclipse：

```
mvn eclipse:eclipse
```

**Step03** 日志 Topology 中的 Tuple 会携带日志域对象，该对象封装了数据和解析日志文件中单个记录的逻辑。在我们的项目中创建这个域对象：

```

public class LogEntry {
    public static Logger LOG = Logger.getLogger(LogEntry.class);
    private String source;
    private String type;
    private List<String> tags = new ArrayList<String>();
    private Map<String,String> fields = new HashMap<String,
        String>();

    private Date timestamp;
    private String sourceHost;
    private String sourcePath;
    private String message = "";
    private boolean filter = false;
    private NotificationDetails notifyAbout = null;
    private static String[] FORMATS = new String[]{"yyyy-MM-
dd'T'HH:mm:ss.SSS",
        "yyyy.MM.dd G 'at' HH:mm:ss z",
        "yyyyy.MMMM.dd GGG hh:mm aaa",
        "EEE, d MMM yyyy HH:mm:ss Z",
        "yyMMddHHmmssZ"};
    @SuppressWarnings("unchecked")
    public LogEntry(JSONObject json){
        source = (String)json.get("@source");
        timestamp =
            parseDate((String)json.get("@timestamp"));
        sourceHost = (String)json.get("@source_host");
        sourcePath = (String)json.get("@source_path");
    }
}

```

```

        message = (String)json.get("@message");
        type = (String)json.get("@type");

        JSONArray array = (JSONArray)json.get("@tags");
        tags.addAll(array);
        JSONObject fields = (JSONObject)json.get("@fields");
        fields.putAll(fields);
    }

    public Date parseDate(String value){
        SimpleDateFormat format = new
            SimpleDateFormat(FORMATS[i]);
        for(int i = 0; i < FORMATS.length; i++){
            Date temp;
            try {
                temp = format.parse(value);
                if(temp != null)
                    return temp;
            } catch (ParseException e) {}
        }
        LOG.error("Could not parse timestamp for log");
        return null;
    }

    @SuppressWarnings("unchecked")
    public JSONObject toJSON(){
        JSONObject json = new JSONObject();
        json.put("@source", source);
        json.put("@timestamp", DateFormat
            .getDateInstance().format(timestamp));
        json.put("@source_host", sourceHost);
        json.put("@source_path", sourcePath);
        json.put("@message", message);
        json.put("@type", type);
        JSONArray temp = new JSONArray();
        temp.addAll(tags);
        json.put("@tags", temp);
        JSONObject fieldTemp = new JSONObject();
        fieldTemp.putAll(fields);
        json.put("@fields", fieldTemp);
        return json;
    } ...

```



这个代码片段中并没有包含 getter、setter 和 equals 方法，但还是有必要依次实现它们。对单元测试来说，equals 方法有很大用处。

**Step04** 接下来创建 Logspout 类，并继承 BaseRichSpout 接口。参照第 1 章的方法实现这个类，声明下面这个字段：

```
outputFieldsDeclarer.declare(new Fields(FieldNames.LOG_ENTRY));
```

然后发送接收到的日志记录到 Topology 中，代码如下：

```
public void nextTuple() {
    String content = jedis.rpop(LOG_CHANNEL);
    if(content==null || "nil".equals(content)) {
        // 休眠，避免发生线程饿死
        try { Thread.sleep(300); }
        catch (InterruptedException e) {}
    } else {
        JSONObject obj=(JSONObject)
            JSONValue.parse(content);
        LogEntry entry = new LogEntry(obj);
        collector.emit(new Values(entry));
    }
}
```



应尽量避免直接在代码中使用字符串。虽说 Tuple 能在运行时高效地处理这些局部变量，但在编译链接的时候，初始化带有字符串元素的代码并没有任何意义。因此，我们还是建议使用静态变量定义代替这种直接使用字符串的方法。

### 2.3.2 解析

Redis Spout 的实现我们已经熟谙于心，本节重点阐述 LogEntry 类域对象中的解析逻辑。logstash 会以单个 JSON 值的形式向 Redis 通道发送日志。这些 JSON 值的格式如下所示：

```
{
  "@source":"file://PATH",
  "@tags":[],
  "@fields":{},
  "@timestamp":"yyyy-MM-ddThh:mm:ss.SSS",
  "@source_host":"hostname",
  "@source_path":"path",
  "@message":"Syslog log line contents",
  "@type":"syslog"
}
```

### 2.3.3 补充内容

JSON 格式包含两种关键结构，分别是 JSON 对象（JSON Object）和 JSON 数组（JSON Array）。JSON 主页（[www.json.org](http://www.json.org)）提供了针对这两种结构的简要定义，为了方便起见，我们就直接在这里罗列出这两种结构的定义。对象是一个无序的名 / 值对（name/value pair）集合。一个对象以 “{”（左括号）开始，以 “}”（右括号）结束。每个名称后跟一个 “:”（冒号），名 / 值对之间使用 “,”（逗号）分隔，如图 2-2 所示。

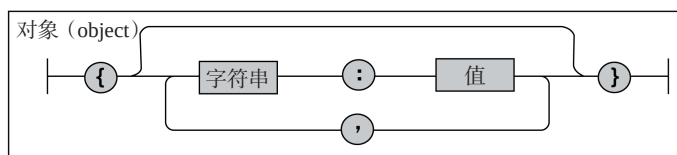


图 2-2 JSON 对象

数组是值 (value) 的有序集合。一个数组以 “[” (左中括号) 开始, 以 “]” (右中括号) 结束。值之间使用 “,” (逗号) 分隔, 如图 2-3 所示。

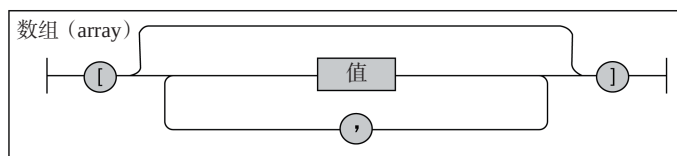


图 2-3 JSON 数组

值 (value) 可以是双引号括起来的字符串 (string)、数值 (number)、true、false、null、对象 (object) 或者数组 (array)。这些结构可以嵌套。

LogEntry 对象的构造函数包含一个 JSONObject 对象作为参数, 并根据该对象的成员对自身进行初始化。可以通过 toJSON() 方法把 LogEntry 对象转换成对应的 JSONObject 对象, 以备不时之需。LogEntry 通过 com.googlecode.json-simple 库中的工具方法, 将字符串转换成可用的 JSON 结构。

虽然结构定义清晰, 但“日期 - 时间”却有多种不同的格式。parseDate() 方法采取最优的方法解析各种日期格式。FORMATS 类成员变量定义了支持的“日期 - 时间”格式。

## 2.4 基于规则的日志流分析

任何合理的日志管理系统都需要具备以下特性:

- (1) 过滤无关紧要的日志信息, 无须对这类日志进行统计和排序。这类日志常常包含 INFO 或 DEBUG 等级的日志记录 (没错, 产品系统中也会包含这些信息)。
- (2) 深入分析日志记录并提取更多有价值的信息和新的字段。
- (3) 在保存日志之前增改日志记录。
- (4) 当收到特定日志记录时发送通知信息。
- (5) 通过关联日志事件来获取有价值的信息。
- (6) 应对日志结构和格式的变更。

本节在 Bolt 中集成了 JBoss 库和 Drools, 以便于我们通过声明和清晰的方式, 轻松实现

上面列举出的这些目标。Drools 是一个基于正向推理的规则引擎的开源实现，它可以推断新的值并根据匹配逻辑执行相关逻辑操作。你可以在这个网站上找到 Drools 项目的更多信息：<http://www.jboss.org/drools/>。

### 2.4.1 实战

**Step01** 使用 Eclipse，在 `storm.cookbook.log` 包中创建 `LogRulesBolt` 类，并继承 `BaseRichBolt`。与 `LogSpout` 类一样，`LogRulesBolt` 类会发送一个包含 `LogEntry` 实例的值到 `Topology` 中。

```
declarer.declare(new Fields(FieldNames.LOG_ENTRY));
```

**Step02** 在 `StatelessKnowledgeSession` 类中添加一个私有成员变量 `ksession`，并在 `Bolt` 的 `prepare` 方法中对其进行初始化。

```
KnowledgeBuilder kbuilder = KnowledgeBuilderFactory.
newKnowledgeBuilder();
kbuilder.add( ResourceFactory.newClassPathResource(
    "/Syslog.drl", getClass() ), ResourceType.DRL );
if ( kbuilder.hasErrors() ) {
    LOG.error( kbuilder.getErrors().toString() );
}
KnowledgeBase kbase = KnowledgeBaseFactory.
newKnowledgeBase();
kbase.addKnowledgePackages( kbuilder.getKnowledgePackages()
);
ksession = kbase.newStatelessKnowledgeSession();
```



该知识会话只初始化了一组 syslog 日志规则。建议把规则管理的工作交给 Drools Guvnor 或者类似的软件，然后通过代理来获取有关规则资源。这个主题已经超出了本书的讨论范围，更多信息可以参考以下链接：<http://www.jboss.org/drools/drools-guvnor.html>。

**Step03** 在 `Bolt` 的 `execute` 方法中实现这个逻辑：把 `Tuple` 中的 `LogEntry` 对象传递到知识会话中。

```
LogEntry entry = (LogEntry)input.getValueByField(FieldNames.LOG_
ENTRY);
if(entry == null){
    LOG.fatal( "Received null or incorrect value from
        tuple" );
    return;
}
ksession.execute( entry );
if(!entry.isFilter()){
    collector.emit(new Values(entry));
}
```

**Step04** 接着创建规则资源文件，使用文本编辑器或 Eclipse 插件就可轻松搞定。可以从更新站点 <http://download.jboss.org/drools/release/5.5.0.Final/org.drools.updatesite/> 下载到相关

Eclipse 插件。规则资源文件应该放置在 classpath 的根目录下。在 src/main/resources 下创建 Syslog.drl，然后右击该目录，依次选择“Build Path”→“Use as source folder”菜单项，将这个目录添加到 Eclipse 的构建路径中。

**Step05** 将以下内容添加到规则资源文件中：

```
package storm.cookbook.log.rules

import storm.cookbook.log.model.LogEntry;
import java.util.regex.Matcher
import java.util.regex.Pattern

rule "Host Correction"

    when
        l: LogEntry(sourceHost == "localhost")
    then
        l.setSourceHost("localhost.example.com");
    end

rule "Filter By Type"
    when
        l: LogEntry(type != "syslog")
    then
        l.setFilter(true);
    end

rule "Extract Fields"
    salience 100// 推迟运行
    when
        l: LogEntry(filter != true)
    then
        String logEntryPattern = "^(([\\d.]+) (\\S+) (\\S+) \\[([\\w:/]+\\s[+\\-]\\d{4})\\] \\\"(.+?)\\\" (\\d{3}) (\\d+)) \\\"([\\^\\"]+)\\\" \\\"([\\^\\"]+)\\\"";
        Matcher matcher = Pattern.compile(logEntryPattern).
matcher(l.getMessage());
        if(matcher.find()){
            l.addField("_pid",matcher.group(1));
            l.addField("_src",matcher.group(2));
        }
    end
```

### 2.4.2 解析

Drools 支持两种类型的知识会话，分别是有状态（stateful）会话和无状态（stateless）会话。在这个例子中，我们只需要一个无状态会话就足够了。



在我们使用有状态会话的时候，需要时刻注意其可能导致的性能问题。有状态会话主要是维持工作会话中内存的“事实”(fact)，这在有些用例中十分重要。但需要了解的是，Drools 所使用的正向推理 Rete 算法引擎的性能还是会随着知识库中“事实”的增加而呈指数级下降。

知识会话根据已知的一组规则来评估事实。Bolt 的 `prepare` 方法中定义了这一系列操作，并在该方法中指定了相关规则。在 Bolt 执行过程中，通过如下调用从 Tuple 中提取 LogEntry 并将其传递到知识会话中：

```
ksession.execute( entry );
```

在执行过程中会把知识会话当做一条记录进行处理，而且该记录在每次调用结束时有可能改变。LogEntry 对象中包含一个名为 `filter` 的控制字段，如果将这个字段设置为 `true`，那么这条日志记录将被丢弃。实现方法是：在规则执行后，发送包含记录的 Tuple 之前，检查这个字段。

```
if(!entry.isFilter()){
    collector.emit(new Values(entry));
}
```

在规则资源文件中，定义了三个规则：

- (1) Host Correction
- (2) Filter By Type
- (3) Extract Fields

这些规则只是为了我们演示一些基本概念而设计的，所以在产品中可能并没有什么用。Host Correction 规则会尝试纠正主机名以使其符合要求。该规则会在满足匹配标准时自动将结果显示出来。该规则的 `when` 子句将 LogEntry 实例的 `sourceHost` 字段与 `localhost` 做匹配操作，当满足条件时触发相关动作。

```
l: LogEntry(sourceHost == "localhost")
```

这个条件子句也会在规则允许的范围内把满足条件的实例赋给局部变量 `l`。`then` 子句是一段传统的 Java 代码 (POJO)，它会在编译后的运行时添加到当前 `classpath` 中。这些规则能让 `localhost` 值满足我们的需求。

```
l.setSourceHost("localhost.example.com");
```

Filter By Type 规则会将所有非 `syslog` 类型日志记录的 `filter` 字段设置成 `true`。

Extract Fields 规则更加有趣。首先，它所包含的 `salience` 命令可以确保其在最后才被执行。这样就可以避免它从已过滤的日志中提取字段。借助于正则表达式匹配，该规则可以从日志文件中提取更多的字段和结构。虽说有关正则表达式的内容超出了本书的讨论范围，但有不少总结得很好文档可供我们参考。

考虑到介绍的完整性，我们在这里罗列了一些有助于删选日志记录的正则表达式示例：

- ❑ 匹配日期：`((?<=))\w+\x20\d+`
- ❑ 匹配时间：`((?<=\\d\\x20))\\d+:\\d+:\\d+`
- ❑ 匹配 IP 地址：`((?<=[,]))(\\d{1,3}\\d{1,3}\\d{1,3}\\d{1,3})`
- ❑ 匹配协议：`((?<=[\\d{1,3}\\d{1,3}\\d{1,3}\\d{1,3}])s))+\\d{1,3}\\d{1,3}\\d{1,3}\\d{1,3}`

可以在维基百科中找到更多有关正则表达式的内容：

[http://en.wikipedia.org/wiki/Regular\\_expression](http://en.wikipedia.org/wiki/Regular_expression)

这些附加字段可以被添加到字段或者标签当中，我们会在稍后的分析、搜索和分组操作中使用这些字段。



Drools 中还有一个叫做 Drools Fusion 的模块，用于支持复杂事件处理（CEP）。CEP 常常被视为一个新的企业级方法，这么讲不为过，但实际上就是指让规则引擎能够在与时间相关的情况下进行正常的处理。它可以根据时间来关联事件，然后导出新的知识或触发相关动作。在本节的 Bolt 实现中就有时序算子的身影。欲了解更多的信息，可以浏览以下链接：

<http://www.jboss.org/drools/drools-fusion.html>

## 2.5 索引与持久化日志数据

我们需要在某些特定时期将日志数据存储起来以便后期利用，而且还要保证这些日志数据能够被检索。为了实现这个目标，本例中将会集成名为 Elastic Search 的开源产品，它是一个通用并集成 RESTful API 的集群搜索引擎（<http://www.elasticsearch.org/>）。

### 2.5.1 实战

**Step01** 创建一个继承自 BaseRichBolt 的 IndexerBolt 类，并声明 org.elasticsearch.client.Client 私有成员变量。你需要在 prepare 方法中初始化它，代码如下：

```
if((Boolean)stormConf.get(backtype.storm.Config.TOPOLOGY_DEBUG) ==
true){
    node = NodeBuilder.nodeBuilder().local(true).node();
} else {
    String clusterName = (String) stormConf.get(Config.ELASTIC_
    CLUSTER_NAME);
    if(clusterName == null)
        clusterName = Config.DEFAULT_ELASTIC_CLUSTER;
    node = NodeBuilder.nodeBuilder().
        clusterName(clusterName).node();
}
client = node.client();
```

**Step02** 然后在 Bolt 的 execute 方法中建立 LogEntry 对象的索引：

```
LogEntry entry = (LogEntry)input.getValueByField(FieldNames.LOG_ENTRY);
if(entry == null){
    LOG.fatal( "Received null or incorrect value from
               tuple" );
    return;
}
String toBeIndexed = entry.toJSON().toJSONString();
IndexResponse response = client.prepareIndex(INDEX_
NAME, INDEX_TYPE)
    .setSource(toBeIndexed)
    .execute().actionGet();
if(response == null)
    LOG.error("Failed to index Tuple: " +
              input.toString());
else{
    if(response.getId() == null)
        LOG.error("Failed to index Tuple: " +
                  input.toString());
    else{
        LOG.debug("Indexing success on Tuple: " +
                  input.toString());
        collector.emit(new Values(entry,response.getId()));
    }
}
```

**Step03** 创建这个 Bolt 的单元测试可能没那么容易，因此值得在这里对此多补充几句。在测试源码目录中的 storm.cookbook.log 包里新建一个 JUnit 4 单元测试，然后添加 StoringMatcher 私有内部类，代码如下：

```
private static class StoringMatcher extends BaseMatcher<Values> {
    private final List<Values> objects = new
ArrayList<Values>();
    @Override
    public boolean matches(Object item) {
        if (item instanceof Values) {
            objects.add((Values) item);
            return true;
        }
        return false;
    }

    @Override
    public void describeTo(Description description) {
        description.appendText("any integer");
    }

    public Values getLastValue() {
        return objects.remove(0);
    }
}
```

**Step04** 然后实现真正的测试逻辑，代码如下：

```
@Test
public void testIndexing() throws IOException {
    // 配置，保证处于调试模式
    Map config = new HashMap();
    config.put(backtype.storm.Config.TOPOLOGY_DEBUG, true);
    Node node = NodeBuilder.nodeBuilder()
        .local(true).node();
    Client client = node.client();

    final OutputCollector collector =
        context.mock(OutputCollector.class);
    IndexerBolt bolt = new IndexerBolt();
    bolt.prepare(config, null, collector);

    final LogEntry entry = getEntry();

    final Tuple tuple = getTuple();
    final StoringMatcher matcher = new StoringMatcher();
    context.checking(new Expectations(){{
        oneOf(tuple).getValueByField(FieldNames.LOG_ENTRY);will
        (returnValue(entry));
        oneOf(collector).emit(with(matcher));
    }});

    bolt.execute(tuple);
    context.assertIsSatisfied();

    // 获取索引的 ID
    String id = (String) matcher.getLastValue().get(1);
    // 检查索引是否正常工作
    GetResponse response =
        client.prepareGet(IndexerBolt.INDEX_NAME
            ,IndexerBolt.INDEX_TYPE,id)
            .execute()
            .actionGet();
    assertTrue(response.exists());
}
```

## 2.5.2 解析

Elastic Search 提供了完整的 Java 客户端 API（它本身也是用 Java 实现的），因此与它集成轻而易举。Bolt 的 prepare 方法会在本地模式或集群模式下创建一个集群节点。集群模式会根据名称获得的集群和在当前节点上创建的本地存储节点连接起来，这样就可以避免在使用不同传输方式进行写操作时发生的双跃点延迟问题。



Elastic Search 本身是一个大型复杂系统，为了更好地理解操作和配置方面的问题，建议你先读一读它所提供的文档。

当 Storm 处于调试模式时，Elastic Search 节点将连同多个在相同 JVM 中被执行的节点（如果需要的话）运行嵌入式集群。这显然对单元测试来说大有裨益。所有这些操作都会在 Bolt 的 prepare 方法中实现。

```
if((Boolean)stormConf.get(backtype.storm.Config.TOPOLOGY_DEBUG) ==
true){
    node = NodeBuilder.nodeBuilder().local(true).node();
} else {
```

当收到 Tuple 时，Bolt 会获取其中的 LogEntry 对象，并将其转化成对应的 JSON 格式内容，然后发送给 Elastic Search。

```
String toBeIndexed = entry.toJSON().toJSONString();
IndexResponse response =
    client.prepareIndex(INDEX_NAME, INDEX_TYPE)
        .setSource(toBeIndexed)
        .execute().actionGet();
```

接着从 Elastic Search 集群的响应（response）中获取日志 ID，并连同 LogEntry 一起发送给下游的 Bolt。在这个例子中，我们只将这个值用于单元测试。不管怎么说，我们都可以很容易地通过添加下游的 Bolt 来持久化这个值，进而将该值用于日志统计信息，这么做能为开发用户界面带来极大便利。

```
collector.emit(new Values(entry,response.getId()));
```

对这个 Bolt 进行单元测试十分需要讲究技巧。这是因为对于一般的单元测试来说，我们在执行单元测试之前就已预知输出结果。但在这个示例中，只有在接收到 Elastic Search 集群的响应后，我们才能知道 ID 的具体值。这使得我们很难提前指定预期输出结果，更别提验证搜索引擎中的日志了。所以为了实现这个目标，我们使用了一个 JMock 自定义匹配器。在 matches 方法中实现了该自定义匹配器的主要逻辑。

```
public boolean matches(Object item) {
    if (item instanceof Values) {
        objects.add((Values) item);
        return true;
    }
    return false;
}
```

该方法能够确保返回 Values 的实例，并将其保存起来。我们会在后续的计算操作用到这个值。有了这个实例，就可以指定以下期望集合：

```
context.checking(new Expectations(){{
    oneOf(tuple).getValueByField(FieldNames
        .LOG_ENTRY);will(returnValue(entry));
    oneOf(collector).emit(with(matcher));
}});
```

并获取记录 ID，根据嵌入式 Elastic Search 集群来验证它。

```
String id = (String) matcher.getLastValue().get(1);
GetResponse response = client.prepareGet(IndexerBolt
    .INDEX_NAME, IndexerBolt.INDEX_TYPE, id)
    .execute()
    .actionGet();
assertTrue(response.exists());
```



若想要实现在集群中搜索日志文件的功能，可以从 kibana.org 下载并安装 Kibana——一款优秀的日志搜索前端引擎。本例通过来自 logstash 的 JSON 日志结构来维护信息，而 Kibana 可作为 Elastic Search 上 logstash 的前端，所以它能与本例所使用的系统进行无缝集成。除此之外，Kibana 还使用了 Twitter Bootstrap 框架，因此你可以非常简单地将其集成到分析面板里。

## 2.6 统计与持久化日志统计信息

我们其实可以收集很多日志流的统计信息，但由于本节只是为了展示一些基本概念，因此我们只准备收集和单时间序列（每分钟）日志量。虽说如此，但这应该已经足以展示有关的设计和方法了，这些技巧能够帮助你在未来实现其他分析工作。

### 2.6.1 实战

**Step01** 下载 storm-cassandra contrib 项目并将其安装到 Maven 仓库中：

```
git clone https://github.com/quintona/storm-cassandra
cd storm-cassandra
mvn clean install
```

**Step02** 在 storm.cookbook.log 包中，创建一个继承自 BaseRichBolt 的类 VolumeCountingBolt。为该 Bolt 声明以下三个字段：

```
public void declareOutputFields(OutputFieldsDeclarer declarer) {
    declarer.declare(new Fields(FIELD_ROW_KEY,
        FIELD_COLUMN, FIELD_INCREMENT));
}
```

**Step03** 然后实现一个静态工具方法，该方法的作用是获取日志的时间信息，按分钟形式表示出来：

```
public static Long getMinuteForTime(Date time) {
    Calendar c = Calendar.getInstance();
    c.setTime(time);
    c.set(Calendar.SECOND, 0);
    c.set(Calendar.MILLISECOND, 0);
```

```

    return c.getTimeInMillis();
}

```

**Step04** 实现 execute 方法（没错，函数体很短）：

```

LogEntry entry = (LogEntry) input.getValueByField(FieldNames.LOG_
ENTRY);
collector.emit(new Values(getMinuteForTime(entry.
getTimestamp()), entry.getSource(),1L));

```

**Step05** 最后，参照第 1 章中的方法来创建 LogTopology 类，并按照如下方法创建 Topology：

```

builder.setSpout("logSpout", new LogSpout(), 10);
builder.setBolt("logRules", new LogRulesBolt(), 10).
shuffleGrouping("logSpout");

builder.setBolt("indexer", new IndexerBolt(), 10).
shuffleGrouping("logRules");
builder.setBolt("counter", new VolumeCountingBolt(), 10).
shuffleGrouping("logRules");
CassandraCounterBatchingBolt logPersistenceBolt = new
    CassandraCounterBatchingBolt(
        Conf.COUNT_CF_NAME, VolumeCountingBolt.FIELD_ROW_KEY,
        VolumeCountingBolt.FIELD_INCREMENT );
    logPersistenceBolt.setAckStrategy
        (AckStrategy.ACK_ON_RECEIVE);
    builder.setBolt("countPersistor",
        logPersistenceBolt, 10).shuffleGrouping("counter");

conf.put(Conf.REDIS_PORT_KEY, Conf.DEFAULT_JEDIS_PORT);
conf.put(CassandraBolt.CASSANDRA_KEYSPACE, Conf.LOGGING_KEYSPACE);

```

## 2.6.2 解析

该实现方法看起来简单得让人感到不可思议，但事实确实如此。在实现过程中我们使用了 storm-cassandra 项目（该项目属于 storm-contrib 组件）来对所有持久化复杂性进行抽象。storm-cassandra 项目通过一个通用且可配置的 backtype.storm.Bolt 实现来集成 Storm 和 Cassandra，该 Bolt 的作用是将 StormTuple 对象写入 Cassandra 列族。

Cassandra 是一个面向列族的数据库系统（<http://cassandra.apache.org/>）。Cassandra 的列族数据模型在提供二级索引<sup>⊖</sup>便利的同时还具备日志式结构更新的存储性能，另外也提供了健壮的物化视图支持和强大的内置缓存。Cassandra 最新的一个功能是计数器列。你可以在集群上的任何地方，通过持久化指定计数器列族中的计数器列来实现计数器的安全自增。

storm-cassandra 项目支持两种持久化风格，第一种是标准的 Cassandra 列族风格，第二种是基于计数器列的风格。我们在这里只关注第二种类型的风格，因为它非常适用于当前这

⊖ 早期版本的Cassandra仅仅支持行键索引（Row Key Index），而现在的版本支持新的列索引（Column Index）。原来的行键索引称为一级索引，而列索引称为二级索引。——译者注

个例子。你可以阅读项目的 README 文件中有关其他风格的介绍，基本上都差不多。

CassandraCounterBatchingBolt 类的实例就能帮我们搞定所有工作。只需告诉它我们要使用哪个列族，哪个 Tuple 字段作为行键，哪个 Tuple 字段作为增量，而 Tuple 中剩下的那个字段则指明了需要更新的计数器列。

我们来看看下面的构造函数：

```
CassandraCounterBatchingBolt logPersistenceBolt = new CassandraCounterBatchingBolt("columnFamily", "RowKeyField", "IncrementAmountField");
```

以及以下作为输入的元组数据：

```
{rowKey: 12345, IncrementAmount: 1L, IncrementColumn: 'SomeCounter'}
```

这会把 columnFamily 列族中的 SomeCounter 计数器列加 1L。

对于任何有关系型数据库背景的开发人员来说，最关键观念转变在于了解面向列族数据库中的数据建模。作为数据库中大数据家族的一分子，面向列族的数据库促进了高度不规则数据模型的应用。这个方法移除了表之间的关系以及它们的锁，实现了对数据库的大规模、并行读写的处理。虽然这种方法会增加数据的冗余，但为了满足当今大规模数据处理的需求，在一些普通磁盘上面的开销其实算不了什么问题。观念的转变在于我们要根据在数据集上执行的查询来选择适当的数据模型，而不是简单地将现实世界模型转化成固有和统一的结构。对应当前查询的数据类型可以归纳为：查询给定时间点上所有日志文件的总数。

这种方法能让我们通过简单地发送 Tuple 来生成统计数据，我们可以使用这种方法来得到各种问题的答案，下面给出几个例子：

- ❑ 我们的日志体积随着给定时间周期（可以是日、月或者年）的变化趋势到底是什么样的？
- ❑ 日志文件中出现频率最高的词干是什么？



列族不只可以包含统计数据，你可以设计任意的不规则结构，并通过发送一个 Tuple 来表示某一行中任意列的集合。如果该行已经存在，Cassandra 会简单地添加或者更新数据。这其实可以实现非常强大的功能。

## 2.7 为日志流集群创建集成测试

在交付过程中，集成测试扮演着非常重要的角色。集成测试的类型有很多。通常来说，单元集成测试和 Topology 集成测试不仅是持续集成构建周期的组成部分，还是对部署集群集成测试的必要功能性风格的补充。我们在这里展示的集成测试方法本质上和第 1 章中的集成测试方法没什么区别，但这里却复杂很多，所以有必要多作些解释。

## 2.7.1 实战

我们先来创建单元测试。

**Step01** 使用 Eclipse，在项目的单元测试源代码目录中的 `storm.cookbook.log` 包里创建一个名为 `IntegrationTestTopology` 的 JUnit 4 单元测试用例。添加一个 `setup` 方法，该方法会在初始化类之前被调用：

```
@BeforeClass
public static void setup() throws Exception {
    setupCassandra();
    setupElasticSearch();
    setupTopology();
}
```

**Step02** 接下来创建 `setup` 方法中所有被调用的初始化方法，我们先来初始化嵌入式版本的 Cassandra：

```
private static void setupCassandra() throws Exception {
    cassandra = new EmbeddedCassandra(9171);
    cassandra.start();
    // 等待一段时间来启动
    Thread.sleep(3000);

    AstyanaxContext<Cluster> clusterContext = new
    AstyanaxContext.Builder()
        .forCluster("ClusterName")
        .withAstyanaxConfiguration(
            new AstyanaxConfigurationImpl()
                .setDiscoveryType(NodeDiscoveryType.NONE))
        .withConnectionPoolConfiguration(
            new ConnectionPoolConfigurationImpl("MyConnectio
nPool")
                .setMaxConnsPerHost(1).setSeeds(
                    "localhost:9171"))
        .withConnectionPoolMonitor(new
    CountingConnectionPoolMonitor())
        .buildCluster(ThriftFamilyFactory.getInstance());

    clusterContext.start();
    Cluster cluster = clusterContext.getEntity();
    KeyspaceDefinition ksDef = cluster.makeKeyspaceDefinition();

    Map<String, String> stratOptions = new HashMap<String,
String>();
    stratOptions.put("replication_factor", "1");
    ksDef.setName(Conf.LOGGING_KEYSPACE)
        .setStrategyClass("SimpleStrategy")
        .setStrategyOptions(stratOptions)
        .addColumnFamily(
            cluster.makeColumnFamilyDefinition().
```

```

setName(Conf.COUNT_CF_NAME)
        .setComparatorType("UTF8Type")
        .setKeyValidationClass("UTF8Type")
        .setDefaultValidationClass("CounterColumnT
ype"));
    cluster.addKeyspace(ksDef);

}

```

**Step03** 然后初始化本地嵌入式 Elastic Search 实例:

```

private static void setupElasticSearch() throws Exception {
    Node node = NodeBuilder.nodeBuilder().local(true).node();
    client = node.client();
    // 等待一段时间, 确保节点启动完成
    Thread.sleep(5000);
}

```

**Step04** 最后, 初始化需要测试的 Topology:

```

private static void setupTopology() {
    // 我们期望所有输出 Tuple 都可以触发执行 Mock 方法, 确保达到
    // 测试目的
    topology.getBuilder().setBolt("testBolt", testBolt, 1)
        .globalGrouping("indexer");
    // 本地模式下运行, 结束时关闭集群

    topology.runLocal(0);
    // 将 jedis 作为集群的输入和输出
    jedis = new Jedis("localhost",
        Integer.parseInt(Conf.DEFAULT_JEDIS_PORT));
    jedis.connect();
    jedis.flushDB();

    // 在运行测试前停顿几秒, 确保初始化完成
    Utils.sleep(5000);
}

```

**Step05** 这样我们就准备好了所有测试 Topology 所需的工具。另外, 我们还需要在测试结束时销毁这些对象和结构体, 在测试套件中增加一个 AfterClass 方法:

```

@AfterClass
public static void shutDown() {
    topology.shutdownLocal();
    jedis.disconnect();
    client.close();
    cassandra.stop();
}

```

**Step06** 然后实现好测试用例就可以大功告成了:

```

@Test
public void inputOutputClusterTest() throws Exception {
    String testData = UnitTestUtils.readFile("/testData1.json");
    jedis.rpush("log", testData);
    LogEntry entry = UnitTestUtils.getEntry();
    long minute = VolumeCountingBolt.getMinuteForTime(entry.
getTimestamp());
    Utils.sleep(6000);
    String id = jedis.rpop(REDIS_CHANNEL);
    assertNotNull(id);
    // 检查索引功能是否工作正常
    GetResponse response = client
        .prepareGet(IndexerBolt.INDEX_NAME,
            IndexerBolt.INDEX_TYPE, id).execute().actionGet();
    assertTrue(response.exists());
    // 检查 Cassandra 中的计数更新是否正常
    AstyanaxContext<Keyspace> astyContext = new AstyanaxContext
        .Builder()
        .forCluster("ClusterName")
        .forKeyspace(Config.LOGGING_KEYSPACE)
        .withAstyanaxConfiguration(
            new AstyanaxConfigurationImpl()
                .setDiscoveryType(NodeDiscoveryType.NONE))
        .withConnectionPoolConfiguration(
            new ConnectionPoolConfigurationImpl(
                "MyConnectionPool")
                .setMaxConnsPerHost(1).setSeeds(
                    "localhost:9171"))
        .withConnectionPoolMonitor(
            new CountingConnectionPoolMonitor())
        .buildKeyspace(ThriftFamilyFactory.getInstance());
    astyContext.start();
    Keyspace ks = astyContext.getEntity();
    Column<String> result = ks.prepareQuery(
        new ColumnFamily<String, String>(
            Config.COUNT_CF_NAME, StringSerializer.get(),
            StringSerializer.get()))
        .getKey(Long.toString(minute)).getColumn(entry.
getSource())
        .execute().getResult();
    assertEquals(1L, result.getLongValue());
}

```

## 2.7.2 解析

该测试用例通过创建嵌入式 Topology 集群实例来实现集成测试功能，其中两个实例分别叫做 Cassandra 和 Elastic Search。然后我们所做的工作就与之前的集成测试一样，将测试数据添加到输入通道，通过 Topology 来处理这些日志记录，并验证搜索引擎中的记录，以及

计数器自增状态正确与否。

 该测试相比普通的单元测试来说需要花费更多时间，因此不应该将它们包含到你的标准 Maven 构建当中。但这个测试有必要作为你在本地开发工作过程当中的重要一环，并在持续集成服务器上进一步进行验证。

## 2.8 创建日志分析面板

日志分析面板是一个通过图形的方式向用户展示统计信息的 Web 应用程序。在实现它之前，我们需要先了解一下用户界面设计原则。

- ❑ **区分焦点：**只显示必要的内容，根据用户试图得到的东西来创建焦点，而不是让用户因无关内容而分心。
- ❑ **极简主义：**从可用性角度考虑，只需要包含必要的图形特性。
- ❑ **响应式处理：**这是一种确保显示内容清晰和一致的方法，无论是在 PC 上还是在平板电脑上进行查看。
- ❑ **基于标准：**你不应该使用非标准的技术进行开发，因为这有可能会妨碍在诸如 iPad 这样的设备上正常访问我们的程序。

本例中的面板会显示每个日志文件每分钟的日志体积动态图。图 2-4 展示了相关的视图输出。

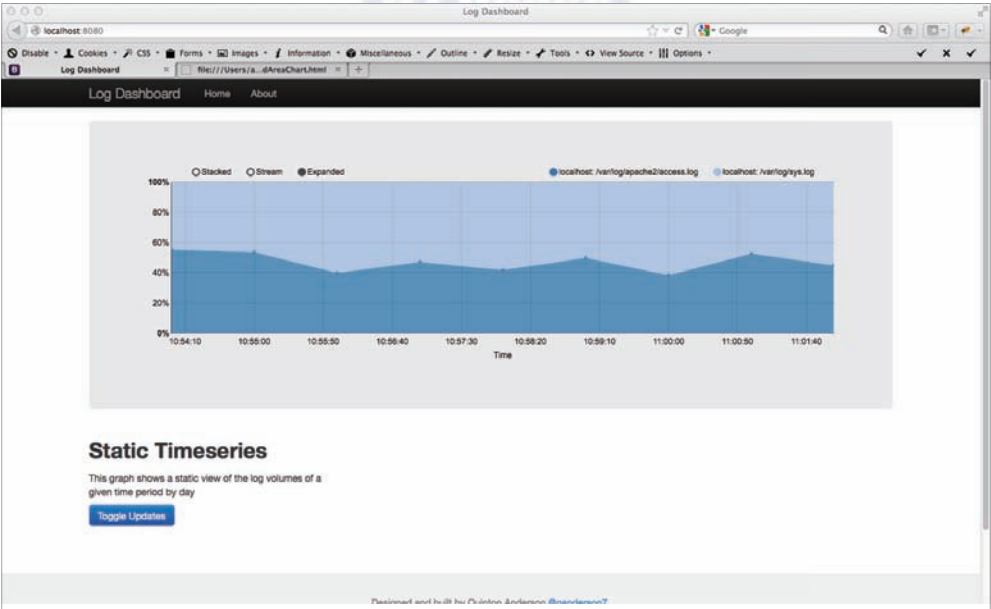


图 2-4 日志体积动态图

图 2-5 展示了详细的监视输出。

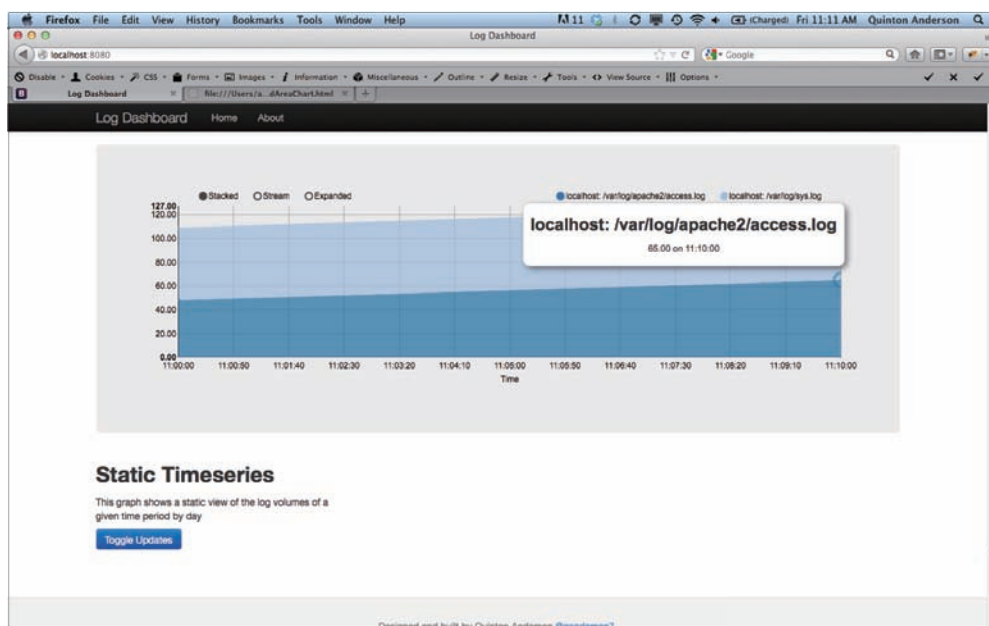


图 2-5 监视输出

## 2.8.1 实战

**Step01** 新建一个名为 log-web 的项目，使用标准 Maven archetype 命令来构建我们的项目：

```
mvn archetype:generate -DgroupId=storm.cookbook -DartifactId=log-web -DarchetypeArtifactId=maven-archetype-webapp
```

该命令会为你生成标准的项目目录结构和 Maven POM 文件。

**Step02** 打开 pom.xml 文件，删除一些默认的依赖项，再添加以下依赖项：

```
<dependency>
  <groupId>junit</groupId>
  <artifactId>junit</artifactId>
  <version>4.8.1</version>
  <scope>test</scope>
</dependency>
<dependency>
  <groupId>org.hectorclient</groupId>
  <artifactId>hector-core</artifactId>
  <version>1.1-2</version>
</dependency>
<dependency>
  <groupId>org.slf4j</groupId>
  <artifactId>slf4j-log4j12</artifactId>
```

```

    <version>1.6.1</version>
  </dependency>
  <dependency>
    <groupId>com.sun.jersey</groupId>
    <artifactId>jersey-server</artifactId>
    <version>1.16</version>
  </dependency>
  <dependency>
    <groupId>com.sun.jersey</groupId>
    <artifactId>jersey-grizzly2</artifactId>
    <version>1.16</version>
  </dependency>
  <dependency>
    <groupId>com.sun.jersey</groupId>
    <artifactId>jersey-servlet</artifactId>
    <version>1.16</version>
  </dependency>
  <dependency>
    <groupId>com.sun.jersey</groupId>
    <artifactId>jersey-json</artifactId>
    <version>1.16</version>
  </dependency>
  <dependency>
    <groupId>com.sun.jersey.contribs</groupId>
    <artifactId>jersey-multipart</artifactId>
    <version>1.16</version>
  </dependency>
  <dependency>
    <groupId>org.jmock</groupId>
    <artifactId>jmock-junit4</artifactId>
    <version>2.5.1</version>
    <scope>test</scope>
  </dependency>
  <dependency>
    <groupId>com.googlecode.json-simple</groupId>
    <artifactId>json-simple</artifactId>
    <version>1.1</version>
  </dependency>

```

**Step03** 然后在 POM 中的 build 区域增加以下构建插件：

```

<plugins>
  <plugin>
    <groupId>org.mortbay.jetty</groupId>
    <artifactId>jetty-maven-plugin</artifactId>
  </plugin>
  <plugin>
    <groupId>org.codehaus.mojo</groupId>
    <artifactId>exec-maven-plugin</artifactId>
    <executions>
      <execution>

```

```

        <goals>
          <goal>java</goal>
        </goals>
      </execution>
    </executions>
  </plugin>
  <plugin>
    <artifactId>maven-compiler-plugin</artifactId>
    <version>2.3</version>
    <configuration>
      <source>1.6</source>
      <target>1.6</target>
      <optimize>true</optimize>
      <showDeprecation>true</showDeprecation>
      <showWarnings>true</showWarnings>
    </configuration>
  </plugin>
  <plugin>
    <groupId>org.codehaus.mojo</groupId>
    <artifactId>cassandra-maven-plugin</artifactId>
  </plugin>
</plugins>

```

**Step04** 通过 `mvn eclipse:eclipse` 命令和 Eclipse 项目导入功能把项目文件导入 Eclipse。

**Step05** 我们会使用优秀的 Twitter Bootstrap GUI 库来创建用户界面。首先将这个库下载到你的独立磁盘分区下，然后将其解压缩。

```

wget http://twitter.github.com/bootstrap/assets/bootstrap.zip
unzip bootstrap.zip

```

**Step06** Bootstrap 提供了许多实例，这有助于我们快速掌握其基本要领。我们先简单复制一份示例，然后再修改它：

```

cp bootstrap/docs/examples/hero.html log-web/src/main/webapp/
index.html
cp bootstrap/docs/about log-web/src/main/webapp/about.html
cp bootstrap/docs/assets log-web/src/main/webapp/
cp bootstrap/docs/assets log-web/src/main/webapp/
cp bootstrap/docs/templates log-web/src/main/webapp/

```



对于开发任何基于 Web 图形用户界面的程序来说，Twitter Bootstrap 都是一个非常好的选择。强烈建议你读一读刚才下载的压缩包中的文档，它可能会给你带来很大的帮助。

**Step07** 有不少 HTML 都需要修改，我们先把重点放在几项关键内容上面：核心展示内容和图形。先更新 `index.html` 文件，用以下代码替换现有的 `<div class="container">` 标签和内容：

```

<div class="container">
  <div class="hero-unit">
    <div id="chart">
      <svg style="height: 300px;"></svg>
    </div>
  </div>
  <div class="row">
    <div class="span4">
      <h2> Timeseries</h2>
      <p>This graph shows a view of the log volumes
        of a given time period by day</p>
      <button id="updateToggleButton" type="button"
        class="btn btn-primary">Toggle Updates</button>
    </div>
  </div>
</div>

```

**Step08** 对于图形来说，我们准备使用一款优秀的可视化库——D3（<http://d3js.org/>），以及一些基于 D3 预先定义好的模型——NVD3（<http://nvd3.org/>），将这些已经编译好的 JavaScript 文件添加到我们的 Web 应用程序的 assets 目录中：

```

wget https://github.com/novus/nvd3/zipball/master
unzip novus-nvd3-4e12985.zip
cp novus-nvd3-4e12985/nv.d3.js log-web/src/main/webapp/assets/js/
cp novus-nvd3-4e12985/lib/d3.v2.js log-web/src/main/webapp/assets/
js/
cp novus-nvd3-4e12985/src/nv.d3.css log-web/src/main/webapp/
assets/css/

```

**Step09** 然后在 HTML 文件中包含这些 JavaScript 文件，并编写用于获取数据和更新图形的客户端 Javascript 代码。

**Step10** 在 HTML 文件末尾的 <script> 标签后面包含以下脚本文件：

```

<script src="assets/js/d3.v2.js"></script>
<script src="assets/js/nv.d3.js"></script>

```

**Step11** 在 html 头部增加 CSS 样式表文件：

```

<link type="text/css" rel="stylesheet" href="assets/css/nv.d3.css">

```

**Step12** 然后在其他导入脚本的代码之后，靠近文件末尾的位置，将以下自定义 JavaScript 代码添加到 <script></script> 标签中：

```

var chart;
var continueUpdates = true;
nv.addGraph(function () {
  chart = nv.models.stackedAreaChart()
    .x(function(d) { return d[0] })
    .y(function(d) { return d[1] })
    .clipEdge(true);

```

```

        chart.xAxis
            .tickFormat(function(d) { return d3.time.
format('%X')(new Date(d)) })
            .axisLabel('Time')
            .showMaxMin(false);

        chart.yAxis
            .axisLabel('Volume')
            .tickFormat(d3.format(',.2f'));

        d3.select('#chart svg')
            .datum(getdata())
            .transition().duration(500)
            .call(chart);

        nv.utils.windowResize(chart.update);

        chart.dispatch.on('stateChange', function (e) {
            nv.log('New State:', JSON.stringify(e));
        });

        return chart;
    });

    function update() {
        fetch();
        if (continueUpdates)
            setTimeout(update, 60000);
    }

    update();

    $(document).ready(function () {
        $('#updateToggleButton').bind('click', function () {
            if (continueUpdates) {
                continueUpdates = false;
            } else {
                continueUpdates = true;
                update();
            }
        });
    });
});

```

**Step13** 添加获取服务器数据的代码:

```

var alreadyFetched = {};

function getUrl(){
    var today = new Date();
    today.setSeconds(0);
    today.setMilliseconds(0);
    var timestamp = today.valueOf();
    var dataurl = "http://localhost:8080/services/LogCount/

```

```

        TotalsForMinute/" + timestamp + "/";
    return dataurl;
}

function fetch() {
    // 通过上面定义的函数来获取 URL
    var dataurl = getUrl();

    // 通过 jQuery 获取数据
    function onDataReceived(series) {
        // 添加到现有数据中
        for(var i = 0; i < series.length; i++){
            if(alreadyFetched[series[i].FileName] == null){
                alreadyFetched[series[i].FileName] = {
                    FileName: series[i].FileName,
                    values: [{
                        Minute: series[i].Minute,
                        Total: series[i].Total
                    }]
                };
            } else {
                alreadyFetched[series[i].FileName].values.push({
                    Minute: series[i].Minute,
                    Total: series[i].Total
                });
                if(alreadyFetched[series[i].FileName].values.
length > 30){
                    alreadyFetched[series[i].FileName].values.
pop();
                }
            }
        }
    }

    // 更新图形
    d3.select('#chart svg')
        .datum(getdata())
        .transition().duration(500)
        .call(chart);
}

function onError(request, status, error){
    console.log("Received Error from AJAX: " +
        request.responseText);
}

$.ajax({
    url:dataurl,
    type:'GET',
    dataType:'json',
    crossDomain: true,
    xhrFields: {
        withCredentials: true
    }
});

```

```

        },
        success: onDataReceived,
        error: onError
    });
}

function getdata(){
    var series = [];
    var keys = [];
    for (key in alreadyFetched) {
        keys.push(key);
    }
    for(var i = 0; i < keys.length; i++){
        var newValues = [];
        for(var j = 0; j < alreadyFetched[keys[i]].values.
length;j++){
            newValues.push([alreadyFetched[keys[i]].values[j].
Minute, alreadyFetched[keys[i]].values[j].Total]);
        }
        series.push({
            key:alreadyFetched[keys[i]].FileName,
            values:newValues
        });
    }
    return series;
}

```

**Step14** 至此，客户端的实现大功告成。为了让客户层能够访问到这些数据，我们还需要为其提供获取数据的服务。

**Step15** 首先，在 `storm.cookbook.services.resources` 包中创建一个工具类 `CassandraUtils`，并添加以下内容：

```

public class CassandraUtils {

    public static Cluster cluster;
    public static Keyspace keyspace;
    protected static Properties properties;

    public static boolean initCassandra(){
        properties = new Properties();
        try {
            properties.load(Main.class
                .getResourceAsStream("/cassandra.properties"));
        } catch (IOException ioe) {
            ioe.printStackTrace();
        }
        cluster = HFactory.getOrCreateCluster(properties.
            getProperty("cluster.name",
                "DefaultCluster"), properties.getProperty
            ("cluster.hosts", "127.0.0.1:9160"));
    }
}

```

```

ConfigurableConsistencyLevel ccl = new
    ConfigurableConsistencyLevel();
ccl.setDefaultReadConsistencyLevel
    (HConsistencyLevel.ONE);

String keyspaceName = properties.getProperty(
    "logging.keyspace", "Logging");

keyspace = HFactory.createKeyspace(
    keyspaceName, cluster, ccl);

return (cluster.describeKeyspace(
    keyspaceName) != null);
}

}

```

**Step16** 在同一包中创建 LogCount 类，用于提供 RESTful 查询服务：

```

@Path("/LogCount")
public class LogCount {

    @GET
    @Path("/TotalsForMinute/{timestamp}")
    @Produces("application/json")
    public String getMinuteTotals(@PathParam("timestamp") String
timestamp){
        SliceCounterQuery<String, String> query =
            HFactory.createCounterSliceQuery(
                CassandraUtils.keyspace,
                StringSerializer.get(),
                StringSerializer.get());
        query.setColumnFamily("LogVolumeByMinute");
        query.setKey(timestamp);
        query.setRange("", "", false, 100);

        QueryResult<CounterSlice<String>> result = query.execute();

        Iterator<HCounterColumn<String>> it =
            result.get().getColumns().iterator();
        JSONArray content = new JSONArray();
        while (it.hasNext()) {
            HCounterColumn<String> column = it.next();
            JSONObject fileObject = new JSONObject();
            fileObject.put("FileName", column.getName());
            fileObject.put("Total", column.getValue());
            fileObject.put("Minute", Long.parseLong(timestamp));
            content.add(fileObject);
        }
        return content.toJSONString();
    }
}

```

**Step17** 最后，通过创建 LogServices 类来提供相关数据服务：

```
@ApplicationPath("/")
public class LogServices extends Application {

    public LogServices(){
        CassandraUtils.initCassandra();
    }
    @Override
    public Set<Class<?>> getClasses() {
        final Set<Class<?>> classes = new HashSet<Class<?>>();
        // 注册根资源
        classes.add(LogCount.class);
        return classes;
    }
}
```

**Step18** 配置 web.xml 文件：

```
<web-app>
  <display-name>Log-Web</display-name>
  <servlet>
    <servlet-name>
      storm.cookbook.services.LogServices</servlet-name>
    <servlet-class>
      com.sun.jersey.spi.container.servlet.ServletContainer
    </servlet-class>
    <init-param>
      <param-name>javax.ws.rs.Application</param-name>
      <param-value>
        storm.cookbook.services.LogServices
      </param-value>
    </init-param>
    <load-on-startup>1</load-on-startup>
  </servlet>
  <servlet-mapping>
    <servlet-name>
      storm.cookbook.services.LogServices</servlet-name>
    <url-pattern>/services/*</url-pattern>
  </servlet-mapping>
</web-app>
```

**Step19** 这时就可以运行你的工程了，在 web-log 项目根目录下执行如下命令：

```
mvn jetty:run
```

然后通过 localhost:8080 访问你的日志分析面板。

## 2.8.2 解析

在上层实现中，面板会根据指定时间周期性地查询服务器。为了能够保存查询结果，它

会在客户端维持一个内存中的结构，并将二维数组转换成图形类。我们来看一下 HTML，以下代码定义了图形显示的位置：

```
<div id="chart">
  <svg style="height: 300px;"></svg>
</div>
```

通过以下代码定义图表：

```
nv.addGraph(function () {
  chart = nv.models.stackedAreaChart()
    .x(function(d) { return d[0] })
    .y(function(d) { return d[1] })
    .clipEdge(true);

  chart.xAxis
    .tickFormat(function(d) { return
      d3.time.format('%X')(new Date(d)) })
    .axisLabel('Time')
    .showMaxMin(false);

  chart.yAxis
    .axisLabel('Volume')
    .tickFormat(d3.format(',.2f'));

  d3.select('#chart svg')
    .datum(getdata())
    .transition().duration(500)
    .call(chart);

  nv.utils.windowResize(chart.update);

  chart.dispatch.on('stateChange', function (e) {
    nv.log('New State:', JSON.stringify(e));
  });

  return chart;
});
```

内存结构其实是一个简单的数值型二维数组，所以我们能够通过以下方法把这些值映射到图形中的 x 和 y 轴：

```
chart = nv.models.stackedAreaChart()
  .x(function(d) { return d[0] })
  .y(function(d) { return d[1] })
  .clipEdge(true);
```

数据是通过 fetch() 方法来获取的，该方法通过 Ajax 异步请求将数据发送给服务器。当客户端接收到响应后，通过 onDataReceived(series) 回调，把相应结果添加到内存结构中。最后，getdata() 方法负责将日志结构映射成对应图形的二维数组。

在服务器端，通过 Jersey 来提供服务。它是一个开源且符合 JSR311 标准的产品，用于构建 RESTful Web 服务。在 Jersey 中，服务通常是通过注解来定义的。本例只需在 LogCount 类中定义一种服务，通过以下注解来实现：

```
@Path("/LogCount")
public class LogCount {

    @GET
    @Path("/TotalsForMinute/{timestamp}")
    @Produces("application/json")
    public String getMinuteTotals(@PathParam("timestamp") String
timestamp){
```

定义好后，这项服务就可以通过 localhost:8080/services/LogCount/TotalForMinutes/[timestamp] 来访问了。我们把传递给 timestamp 变量的值作为查询条件，来执行 Cassandra 查询。最终的查询结果会被映射到 JSON 对象上，然后将其返回给调用者：

```
Iterator<HCounterColumn<String>> it = result.get().getColumns().
iterator();
    JSONArray content = new JSONArray();
    while (it.hasNext()) {
        HCounterColumn<String> column = it.next();
        JSONObject fileObject = new JSONObject();
        fileObject.put("FileName", column.getName());
        fileObject.put("Total", column.getValue());
        fileObject.put("Minute", Long.parseLong(timestamp));
        content.add(fileObject);
    }
    return content.toJSONString();
```



通常来讲，我们不会调用整个 Topology 和集群，这是为了确保测试 Web 应用程序的简单性。在支持材料中提到了一个实用的 main 类，它可以使用随机数据来填充列族，这样就可以对 Web 应用程序进行简单独立的测试了。



# BIG DATA



## 大数据技术丛书

